

QUBIT

Quantum Upgrade Bridge & Inventory Tool — Paper Evidence Pack

field	value
commit	63823b49e399
branch	main
uncommitted files	112
built (UTC)	2026-08-23T18:14:22+00:00
generated by	paper_evidence/scripts/*.py

Every number in this pack traces to a CSV in `tables/` or `data/`, or to the command logged beside it. Values that could not be measured are written UNKNOWN and listed in GAPS with the action that would resolve them. No figure in this document is an estimate.

1. Architecture

QUBIT is a native Windows desktop application over a local HTTP engine. Nothing it does requires a network service the user did not name: discovery, risk, transformation and verification all run on the machine, against a local model, writing to one SQLite file. That is a structural property rather than a policy — the engine binds to `127.0.0.1`, and the shell bundles the dashboard instead of serving it.

Layers

Interface. A Tauri 2 shell hosts the React dashboard; `qubit-cli` offers the same operations without a window, for scripting and for reproducing a run from a shell.

Service. `qubit-api` is a FastAPI application bound to loopback, with bearer-token authentication and rate limiting. It is the only writer the interface layer talks to.

Engine. Four packages, none of which knows about the others except through the inventory:

- `qubit-scanner` — 6 independent discovery sources (AST, configuration, live TLS, certificates, dependency manifests, HashiCorp Vault), driven by a data-only rule catalog: **268 rules compiled into 307 language bindings across 19 tree-sitter grammars**, plus a package→algorithm map covering

903 packages.

- `qubit-risk` — CRQC Monte-Carlo, Mosca inequality, CNSA 2.0 milestone evaluation.
- `qubit-migrate` — plan, patch, and a validation gate that must pass before a patch is offered.
- `qubit-bridge` — hybrid TLS handshake verification.

Foundation. `qubit-core` owns the frozen CryptoAsset schema, the canonical registry of **150 algorithms**, the database, CycloneDX 1.7 CBOM export, and report rendering.

Every other package depends on it and it depends on none of them at run time.

The one edge that looks like a cycle

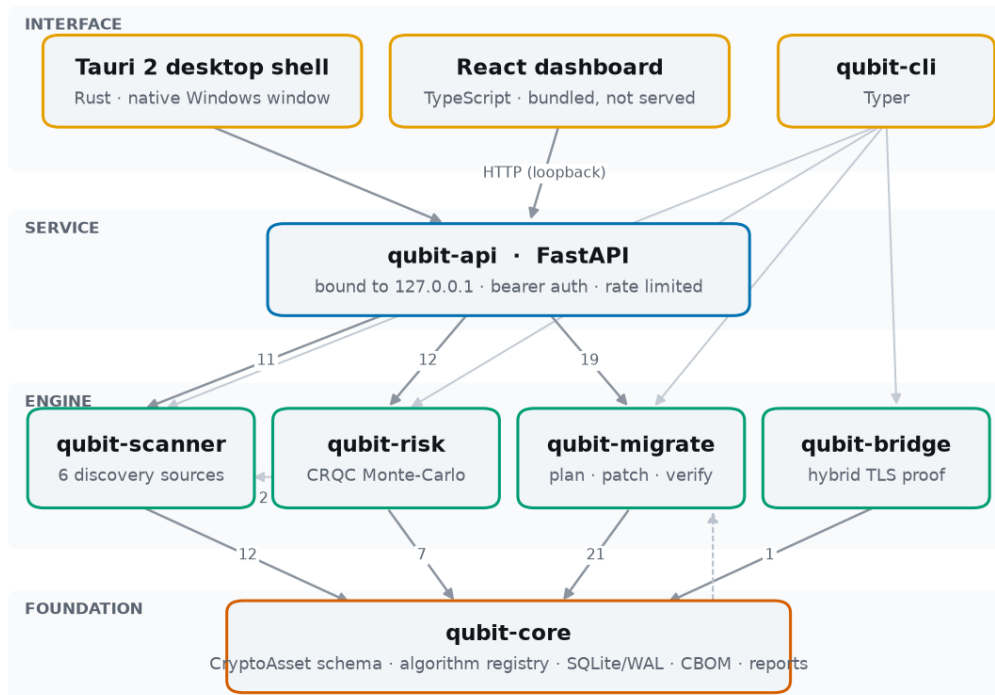
`qubit-core/alembic/env.py` imports the migration models so a single Alembic `target_metadata` sees all 12 tables. It is an import for schema registration, not a runtime dependency, and F02 draws it dotted for that reason. The 171 runtime edges point strictly downward.

Rules are data, not code

The detection catalog is YAML validated against a `qubit-rule/v1` schema, not Python. A new rule is a file, and every rule ships its own positive and negative fixtures which run as tests. This is why the rule set can be counted, versioned and audited — and why the counts above are read from the catalog at build time rather than typed into this document.

Storage

One SQLite database in WAL mode, 12 tables (F05). The inventory is the single source of truth: risk and migration annotate the same assets row rather than keeping a second copy, so a finding cannot be current in one view and stale in another.



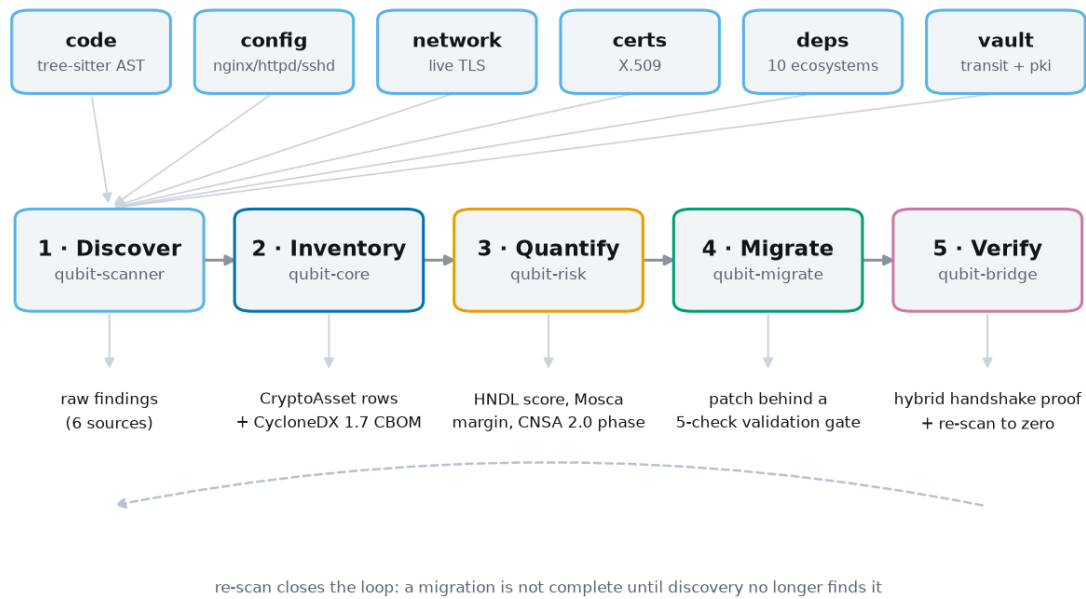
numbers are import-statement counts · dashed = Alembic metadata registration, not a runtime dependency · every edge: tables/T_imports.csv

F02_architecture. QUBIT is four layers over seven Python packages, drawn from the 171 real inter-package import statements rather than from a design document. Every runtime edge points downward; the 2 upward ones live in qubit-core/alembic/env.py and exist so Alembic sees all twelve tables under one metadata object, which is schema registration and not a runtime dependency. Note that qubit-cli reaches the engine packages directly rather than through the API, so a scripted run and a windowed run are not the same path. The desktop shell bundles the dashboard rather than serving it, and the engine binds to 127.0.0.1 — which is what makes the offline claim structural rather than a policy. Source: tables/T_imports.csv.

T03 — technology stack, with resolved versions

layer	component	technology	version	role
Interface	Desktop shell	Tauri	2.x	native Windows window over a local engine
Interface	Dashboard	React + TypeScript	UNKNOWN	bundled into the shell, not served over a network
Interface	Command line	Typer	0.27.0	scriptable entry point
Service	HTTP engine	FastAPI	0.139.2	bound to 127.0.0.1, bearer auth, rate limited
Service	ASGI server	uvicorn	0.51.0	single local process
Engine	Source parsing	tree-sitter	0.25.2	AST queries over 19 grammars
Engine	Certificates / TLS	cryptography	49.0.0	X.509 parsing and handshake primitives
Engine	Transformation	Ollama (qwen2.5-coder:7b)	UNKNOWN	local inference, greedy decoding, no network egress
Engine	Dependency graph	networkx	3.6.1	migration ordering over unit dependencies
Foundation	ORM	SQLAlchemy	2.0.51	12 tables, single metadata
Foundation	Migrations	Alembic	1.18.5	versioned schema
Foundation	Storage	SQLite (WAL)	UNKNOWN	one file, no server
Foundation	Validation	Pydantic	2.13.4	frozen CryptoAsset schema
Foundation	Reports	reportlab	5.0.0	pure Python, no system libraries
Evidence	Figures	matplotlib	3.11.1	vector SVG, Okabe-Ito
Evidence	Statistics	scipy	1.18.0	exact McNemar, Wilson, log-linear fits

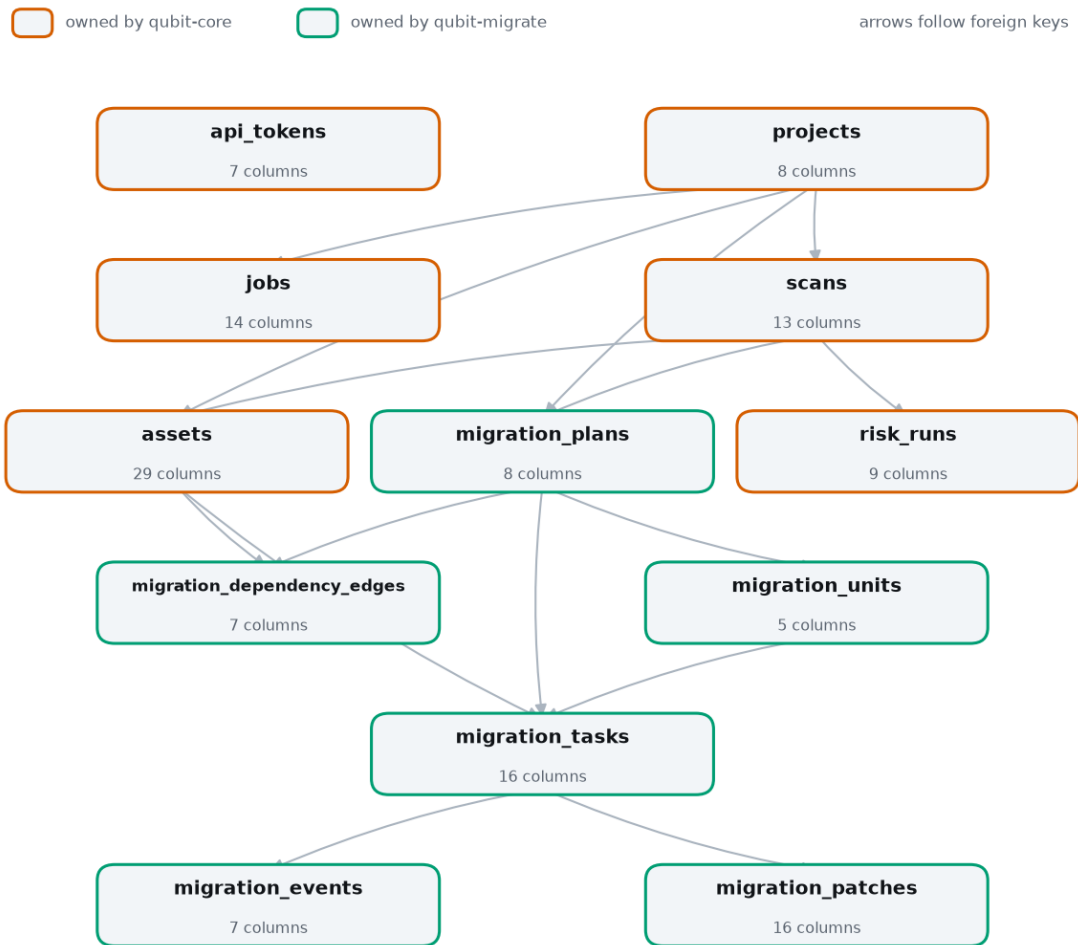
2. Workflow



F03_workflow. The pipeline, with the module that implements each stage and the artifact it leaves behind. Six independent discovery sources feed one normalised inventory, so a finding from a TLS handshake and a finding from an AST are the same kind of object by the time risk is computed. The loop is closed rather than open: a migration counts as complete only when a fresh scan no longer reports the finding, which is what stage 5 verifies. Every stage is local — no step contacts a network service the user did not name.

Each stage writes into the same inventory rather than into a private store, so the artifact one stage produces is the input the next one reads. The truth tables in section 6 give the exact decision rules for the two stages that make a judgement call: screening a finding, and admitting a patch.

3. Data model



F05_data_model. The physical schema, introspected from SQLAlchemy metadata rather than transcribed: 12 tables and 139 columns, laid out by foreign-key depth. The frozen CryptoAsset schema is flattened into the filterable columns of `assets`, with the structured remainder in JSON, and risk and migration write their annotations back onto the same row rather than forking a second copy of the truth. Migration owns six tables of its own but reaches the inventory only through foreign keys. Full column listing: `tables/T_schema.csv`.

T_schema — every column, introspected

table	owner	column	type	nullable	primary_key	foreign_key
api_tokens	qubit-core	id	CHAR(32)	False	True	
api_tokens	qubit-core	name	VARCHAR(64)	False	False	
api_tokens	qubit-core	token_hash	VARCHAR(64)	False	False	
api_tokens	qubit-core	scopes	VARCHAR(2)	False	False	
api_tokens	qubit-core	created_at	DATETIME	False	False	
api_tokens	qubit-core	last_used_at	DATETIME	True	False	
api_tokens	qubit-core	revoked_at	DATETIME	True	False	
assets	qubit-core	id	CHAR(32)	False	True	
assets	qubit-core	scan_id	CHAR(32)	False	False	scans.id
assets	qubit-core	project_id	CHAR(32)	False	False	projects.id
assets	qubit-core	fingerprint	VARCHAR(16)	False	False	
assets	qubit-core	source_scanner	VARCHAR(8)	False	False	
assets	qubit-core	asset_type	VARCHAR(16)	False	False	
assets	qubit-core	algorithm	VARCHAR(64)	False	False	
assets	qubit-core	key_size	INTEGER	True	False	
assets	qubit-core	usage_context	VARCHAR(20)	False	False	

table	owner	column	type	nullable	primary_key	foreign_key
assets	qubit-core	sensitivity	VARCHAR(12)	False	False	
assets	qubit-core	shelf_life_years	FLOAT	True	False	
assets	qubit-core	qv_vulnerable	BOOLEAN	False	False	
assets	qubit-core	qv_attack	VARCHAR(8)	False	False	
assets	qubit-core	confidence	VARCHAR(8)	False	False	
assets	qubit-core	rule_id	VARCHAR	True	False	
assets	qubit-core	stale	BOOLEAN	False	False	
assets	qubit-core	location	JSON	False	False	
assets	qubit-core	protocol_detail	JSON	True	False	
assets	qubit-core	library	JSON	True	False	
assets	qubit-core	evidence	JSON	False	False	
assets	qubit-core	discovered_at	DATETIME	False	False	
assets	qubit-core	last_seen_at	DATETIME	False	False	
assets	qubit-core	risk_score	FLOAT	True	False	
assets	qubit-core	risk_ci_low	FLOAT	True	False	
assets	qubit-core	risk_ci_high	FLOAT	True	False	
assets	qubit-core	mosca_margin_years	FLOAT	True	False	
assets	qubit-core	priority_rank	INTEGER	True	False	
assets	qubit-core	migration_status	VARCHAR(10)	True	False	
assets	qubit-core	migration_json	JSON	True	False	
jobs	qubit-core	id	CHAR(32)	False	True	
jobs	qubit-core	kind	VARCHAR(16)	False	False	
jobs	qubit-core	status	VARCHAR(12)	False	False	
jobs	qubit-core	project_id	CHAR(32)	True	False	projects.id
jobs	qubit-core	ref_id	CHAR(32)	True	False	
jobs	qubit-core	progress	FLOAT	False	False	
jobs	qubit-core	stage	VARCHAR(64)	False	False	
jobs	qubit-core	message	VARCHAR(256)	False	False	
jobs	qubit-core	payload	JSON	False	False	

(94 further rows in T_schema.csv)

T_imports — the inter-package edges behind F02

from_package	to_package	import_statements	kind
qubit_api	qubit_core	38	runtime
qubit_api	qubit_migrate	19	runtime
qubit_api	qubit_risk	12	runtime
qubit_api	qubit_scanner	11	runtime
qubit_bridge	qubit_core	1	runtime
qubit_cli	qubit_bridge	4	runtime
qubit_cli	qubit_core	22	runtime
qubit_cli	qubit_migrate	3	runtime
qubit_cli	qubit_risk	11	runtime
qubit_cli	qubit_scanner	8	runtime
qubit_core	qubit_migrate	1	alembic metadata only
qubit_core	qubit_risk	1	alembic metadata only
qubit_migrate	qubit_core	21	runtime
qubit_risk	qubit_core	7	runtime
qubit_risk	qubit_scanner	2	runtime
qubit_scanner	qubit_core	12	runtime

4. Module inventory and languages

Lines of code per module.

module	files	physical_lines	code_lines
benchmarks	57	28401	26543
packages/qubit-scanner	95	23711	20829
packages/qubit-migrate	69	11192	8617
dashboard	56	9579	7959
packages/qubit-api	34	7560	5856
docs	17	8211	5811
packages/qubit-risk	48	5104	4014
packages/qubit-core	35	4977	3819
packages/qubit-cli	13	2906	2268
packages/qubit-bridge	18	1782	1405
(root)	7	710	498
.github	1	121	109
scripts	1	81	60

By language, source separated from config and docs.

language	category	files	physical_lines	code_lines
Python	source	259	45475	35106
YAML	config/docs	75	23549	21967
JSON	config/docs	31	15860	15852
TypeScript	source	42	8420	7008
Markdown	config/docs	28	9687	6786
CSS	config/docs	1	592	517
TOML	config/docs	10	330	262
Rust	source	2	331	222
Shell	source	2	75	53
HTML	config/docs	1	16	15

Declared test functions per area.

area	test_functions
packages/qubit-scanner	175
packages/qubit-migrate	159
packages/qubit-api	110
benchmarks	84
packages/qubit-risk	69
packages/qubit-core	64
packages/qubit-cli	40
packages/qubit-bridge	23

Every configuration setting and its default.

source	setting	type	default
packages/qubit-api/src/qubit_api/settings.py	db_url	str	Field(default_factory=default_db_url)
packages/qubit-api/src/qubit_api/settings.py	api_token	str	'qubit-dev-token-do-not-use-in-prod' # noqa: S105 — dev default; override via QUBIT_API_TOKEN
packages/qubit-api/src/qubit_api/settings.py	api_prefix	str	'/api/v1'
packages/qubit-api/src/qubit_api/settings.py	bind_host	str	'127.0.0.1'

source	setting	type	default
packages/qubit-api/src/qubit_api/settings.py	bind_port	int	8787
packages/qubit-api/src/qubit_api/settings.py	create_schema_on_startup	bool	True
packages/qubit-api/src/qubit_api/settings.py	dashboard_dist	str None	None
packages/qubit-api/src/qubit_api/settings.py	scan_roots	str	"
packages/qubit-api/src/qubit_api/settings.py	rate_limit_per_minute	int	120
packages/qubit-migrate/src/qubit_migrate/config.py	model	str	'qwen2.5-coder:7b-instruct-q4_K_M'
packages/qubit-migrate/src/qubit_migrate/config.py	fallback_model	str	'qwen2.5-coder:1.5b-instruct-q4_K_M'
packages/qubit-migrate/src/qubit_migrate/config.py	llm_timeout	float	180.0
packages/qubit-migrate/src/qubit_migrate/config.py	llm_verify_rescan	bool	True
packages/qubit-migrate/src/qubit_migrate/config.py	max_repair_rounds	int	2
packages/qubit-migrate/src/qubit_migrate/config.py	min_confidence	float	0.5
packages/qubit-migrate/src/qubit_migrate/config.py	no_docker	bool	False # set True to skip stages 3-4
packages/qubit-risk/src/qubit_risk/config.py	hardware_priors	dict[str, Any]	(required)
packages/qubit-risk/src/qubit_risk/config.py	resource_estimates	dict[str, Any]	(required)
packages/qubit-risk/src/qubit_risk/config.py	sensitivity_rules	dict[str, Any]	(required)
packages/qubit-risk/src/qubit_risk/config.py	shelf_life_priors	dict[str, Any]	(required)
packages/qubit-risk/src/qubit_risk/config.py	mosca	dict[str, Any]	(required)
packages/qubit-risk/src/qubit_risk/config.py	expert_survey	dict[str, Any]	field(default_factory=dict)
packages/qubit-risk/src/qubit_risk/config.py	bn_cpds	dict[str, Any]	field(default_factory=dict)
packages/qubit-risk/src/qubit_risk/config.py	cnsa2_milestones	dict[str, Any]	field(default_factory=dict)
packages/qubit-risk/src/qubit_risk/config.py	params_hash	str	field(default="")

(1 further rows in T_config.csv)

5. Public API and CLI surface

A9 — Public API and CLI surface

REST routes (51)

method	path	defined in
DELETE	``	packages/qubit-api/src/qubit_api/routers/projects.py
GET	``	packages/qubit-api/src/qubit_api/routers/projects.py
POST	``	packages/qubit-api/src/qubit_api/routers/projects.py
GET	/algorithms	packages/qubit-api/src/qubit_api/routers/registry.py
GET	/assets/{asset_id}	packages/qubit-api/src/qubit_api/routers/assets.py
GET	/assets/{asset_id}/hndl	packages/qubit-api/src/qubit_api/routers/risk.py
GET	/assets/{asset_id}/recommendation	packages/qubit-api/src/qubit_api/routers/recommendation.py
GET	/auth/whoami	packages/qubit-api/src/qubit_api/auth.py
GET	/events	packages/qubit-api/src/qubit_api/routers/jobs.py
GET	/health	packages/qubit-api/src/qubit_api/routers/meta.py
GET	/health/deps	packages/qubit-api/src/qubit_api/routers/meta.py
GET	/jobs	packages/qubit-api/src/qubit_api/routers/jobs.py
GET	/jobs/{job_id}	packages/qubit-api/src/qubit_api/routers/jobs.py
POST	/jobs/{job_id}/cancel	packages/qubit-api/src/qubit_api/routers/jobs.py
GET	/jobs/{job_id}/events	packages/qubit-api/src/qubit_api/routers/jobs.py
GET	/languages	packages/qubit-api/src/qubit_api/routers/registry.py
GET	/meta/agility-policy	packages/qubit-api/src/qubit_api/routers/meta.py
GET	/meta/migration-kb	packages/qubit-api/src/qubit_api/routers/meta.py
POST	/migrate/patches/{patch_id}/apply	packages/qubit-api/src/qubit_api/routers/migrate.py
POST	/migrate/patches/{patch_id}/review	packages/qubit-api/src/qubit_api/routers/migrate.py
GET	/migrate/plans	packages/qubit-api/src/qubit_api/routers/migrate.py
POST	/migrate/plans	packages/qubit-api/src/qubit_api/routers/migrate.py
GET	/migrate/plans/{plan_id}/graph	packages/qubit-api/src/qubit_api/routers/migrate.py
GET	/migrate/plans/{plan_id}/queue	packages/qubit-api/src/qubit_api/routers/migrate.py
POST	/migrate/plans/{plan_id}/run	packages/qubit-api/src/qubit_api/routers/migrate.py
POST	/migrate/tasks/{task_id}/advise	packages/qubit-api/src/qubit_api/routers/migrate.py
POST	/migrate/tasks/{task_id}/generate	packages/qubit-api/src/qubit_api/routers/migrate.py

method	path	defined in
GET	/migrate/tasks/{task_id}/governance	packages/qubit-api/src/qubit_api/routers/migrate.py
GET	/migrate/tasks/{task_id}/patches	packages/qubit-api/src/qubit_api/routers/migrate.py
GET	/overview	packages/qubit-api/src/qubit_api/routers/projects.py
GET	/risk/runs/{risk_run_id}	packages/qubit-api/src/qubit_api/routers/risk.py
GET	/risk/timeline	packages/qubit-api/src/qubit_api/routers/risk.py
DELETE	/scans	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans	packages/qubit-api/src/qubit_api/routers/scans.py
DELETE	/scans/{scan_id}	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans/{scan_id}	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans/{scan_id}/assets	packages/qubit-api/src/qubit_api/routers/assets.py
GET	/scans/{scan_id}/cbom	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans/{scan_id}/cnsa2	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans/{scan_id}/diff	packages/qubit-api/src/qubit_api/routers/scans.py
POST	/scans/{scan_id}/risk/run	packages/qubit-api/src/qubit_api/routers/risk.py
GET	/scans/{scan_id}/risk/summary	packages/qubit-api/src/qubit_api/routers/risk.py
GET	/scans/{scan_id}/risk/timeline	packages/qubit-api/src/qubit_api/routers/risk.py
GET	/scans/{scan_id}/sarif	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/scans/{scan_id}/summary	packages/qubit-api/src/qubit_api/routers/scans.py
GET	/version	packages/qubit-api/src/qubit_api/routers/meta.py
DELETE	/ {project_id}	packages/qubit-api/src/qubit_api/routers/projects.py
GET	/ {project_id}	packages/qubit-api/src/qubit_api/routers/projects.py
PATCH	/ {project_id}	packages/qubit-api/src/qubit_api/routers/projects.py
GET	/ {project_id}/scans	packages/qubit-api/src/qubit_api/routers/projects.py
GET	/ {project_id}/trends	packages/qubit-api/src/qubit_api/routers/projects.py

CLI commands (5)

command	entry point
qubit report	qubit-cli
qubit scan	qubit-cli
qubit scan-network	qubit-cli
qubit scan-vault	qubit-cli
qubit version	qubit-cli

6. Decision logic — truth tables

T12_screening_classifier — Screening classifier (benchmarks/oracles/adjudicate.py::classify)

The classifier the corpus-level use/mention result rests on, run on real corpus lines. Rows 8-10 are the ones an earlier version got wrong: it stripped non-alphanumerics before searching, so 3DES matched inside CODES, DESC and SITE_DESCRIPTION, and 240 non-cryptographic lines on one repository were scored as real code. Measured agreement with hand labels is in benchmarks/adjudication/.

case	line	family	classification
real call	mac := hmac.New(<code>sha256.New</code> , key)	SHA	code
real call, digit boundary	<code>hashlib.md5(b'x')</code>	MD5	code
constructor	<code>des.NewTripleDESCipher(key)</code>	3DES	code
quoted argument	<code>Cipher.getInstance("DESede/CBC/PKCS5Padding")</code>	3DES	string-literal
string literal only	<code>banned = ['RC4', 'MD5']</code>	RC4	string-literal
whole-line comment	<code>// ARC4 is a stream cipher</code>	RC4	comment
trailing comment	<code>x = 1 # uses MD5 historically</code>	MD5	comment
substring in identifier	<code>id: CODES.BadResponse,</code>	3DES	substring
substring, SQL	<code>ORDER BY created_at DESC</code>	3DES	substring
substring, camelCase	<code>const SITE_DESCRIPTION = 'hi'</code>	3DES	substring
name absent entirely	<code>int main(void) { return 0; }</code>	RSA	substring
category it will not judge	<code>password = "hunter2"</code>	HARDCODED PASSWORD/SECRET	not-applicable
empty line	<code>(empty)</code>	MD5	not-applicable
empty algorithm	<code>hashlib.md5(b'x')</code>	(empty)	not-applicable

T12_site_collapse — Overlapping-rule reconciliation (qubit_scanner.code.scanner::_one_per_site)

One call is one cryptographic fact however many rules recognise it. The catalog deliberately overlaps — a generic `Signature.getInstance(<alg>)` rule for coverage and a specific ML-DSA rule that carries the migration example — and before this both reached the inventory, inflating every count taken over it. Specificity is the number of where constraints; ties break on confidence and then rule id, so the inventory does not depend on catalog iteration order. Comparison is on the **resolved** algorithm, so Dilithium3 and ML-DSA-65 collapse together.

case	rule A constraints	A confidence	rule B constraints	B confidence	kept
generic vs PQC-specific, same name	2	high	3	high	the specific rule
generic vs PQC-specific, alias name	2	high	3	high	the specific rule
same specificity, different confidence	3	medium	3	high	the higher confidence
identical on both	3	high	3	high	the lower rule id (deterministic)
more constrained but lower confidence	2	high	4	low	the more constrained rule

T12_sonar_coverage_gate — Fifth-detector applicability gate (benchmarks/oracles/sonar_oracle.py)

sonar-cryptography reads Java, Python and Go. A repository is scanned only if it has ≥ 25 analysable files **and** they are $\geq 5\%$ of it. Both halves are load-bearing: openwrt passes the count on 25 build scripts and fails the share, and scanning it would cost an hour to produce a guaranteed zero. A skip is recorded as 'cannot read', which is a different statement from 'found nothing'.

repository	analysable	total files	share	$\geq$ files	$\geq$ share	verdict
RxJava	2073	2184	94.9%	yes	yes	scan
TheAlgorithms/Python	1385	1537	90.1%	yes	yes	scan
conductor	1486	4171	35.6%	yes	yes	scan
CymChad (mixed Kotlin)	25	263	9.5%	yes	yes	scan
openwrt (build scripts)	25	11555	0.2%	yes	no	skip
redis (build scripts)	48	1887	2.5%	yes	no	skip
libuv	3	509	0.6%	no	no	skip
SwiftLint	0	988	0.0%	no	no	skip

T12_validation_gate — Patch validation gate (qubit_migrate.transform.validate)

All 243 combinations of five stages $\times$ {pass, fail, skipped} were evaluated. The verdict depends on only two things, and the collapse below is asserted sound against all 243. **A skipped stage never turns a failure into a pass, but it does mean the patch was not fully verified** — which is what `partial` reports, and what a caller must not ignore.

any stage failed	any stage skipped	passed	partial	meaning
no	no	True	False	accepted and fully verified
no	yes	True	True	accepted, but not fully verified
yes	no	False	False	patch rejected
yes	yes	False	True	patch rejected

7. Environment and dependencies

hardware.txt

```
# A11 -- Environment

captured_utc: 2026-08-23T18:13:47+00:00
platform: Windows-11-10.0.26200-SP0
machine: AMD64
processor: Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
python: 3.12.13 (CPython)
logical_cpus: 28
total_memory_bytes: 16873545728
cpu_name: Intel(R) Core(TM) i7-14700HX

## Git

commit: 63823b49e3991c31176b66a8cb66fab4e25a563b
branch: main
dirty_files: 112

## Container image digests (benchmark oracles)

qubit-bench-cryptoscan:11f0e46: qubit-bench-cryptoscan@sha256:0017535511487f7f5eecd706cc429ddb346c179563013cc0
semgrep/semgrep:latest: semgrep/semgrep@sha256:67319956da3dcb58baf5b322899c15458e3963e7018a86aeeb5cd224e69cb77
sonarqube:community: sonarqube@sha256:9026624a61cd25542a402a9e7213dd7dbb39724ac9597e331e6b85362558c079
sonarsource/sonar-scanner-cli:latest: sonarsource/sonar-scanner-cli@sha256:23ca0f137965d9dff2198074043fd48d386
hashicorp/vault:latest: hashicorp/vault@sha256:5be49781ecf78bfe775c5309c6a4d9f4e9e040b6c885c99eb2b12fb69855e1a
```

versions.lock

```
# Resolved environment

python: 3.12.13

## Python (pip freeze)

## Declared per package

qubit-api:
fastapi>=0.139
pydantic-settings>=2.6
qubit-core
qubit-migrate
qubit-risk
qubit-scanner
sse-starlette~=3.4
uvicorn>=0.35
qubit-bridge:
qubit-core
typer>=0.12
```

pydantic>=2.7
httpx>=0.27
pyshark>=0.6
jinja2>=3.1
qubit-cli:
qubit-core
qubit-scanner
qubit-api
qubit-risk
qubit-migrate
qubit-bridge
alembic>=1.13
typer>=0.12
rich>=13
qubit-core:
pydantic>=2.7
sqlalchemy>=2.0.30
alembic>=1.13
platformdirs>=4.2
qubit-migrate:
qubit-core
networkx>=3.3
libcst>=1.4
tree-sitter>=0.24,<0.26
tree-sitter-language-pack>=1.12
GitPython>=3.1.43
PyYAML>=6.0.1
typer>=0.12
rich>=13.7
unidiff>=0.7.5
pydantic-settings>=2.0
qubit-risk:
qubit-core
numpy>=2.0
scipy>=1.13
pyyaml>=6
pgmpy>=1.0
qubit-scanner:
qubit-core

8. Model card

Model card — migration code transformer

The only learned component in QUBIT. Everything else (detection, risk, CBOM) is rule-based, so there is exactly one of these cards. Fields the repository cannot settle are UNKNOWN and appear in GAPS.md with the action that would resolve them.

Task

field	value
task	Rewrite one source file so a named quantum-vulnerable algorithm is replaced by its post-quantum or hardened equivalent, preserving behaviour
input_repr	Whole source file as text, plus the migration rule, the asset's algorithm and line, and a verified target shape quoted from the scanner's own rule examples
output_repr	Whole rewritten file, fenced; a unified diff is derived from it, never requested directly
pretrained_or_scratch	Pretrained, used as-is
finetune_method	None. No training, no LoRA, no adapters. This is prompt-driven inference against a stock checkpoint

Model

field	value
model_family	Qwen2.5-Coder (instruct)
checkpoint	qwen2.5-coder:7b-instruct-q4_K_M
fallback_checkpoint	qwen2.5-coder:1.5b-instruct-q4_K_M
quantisation	Q4_K_M (4-bit, k-quant medium)
serving	Ollama, local HTTP, no network egress
version_or_commit	Ollama tag as above; blob digest dae161e27b0e90dd1856c8bb3209201fd6736d8eb66298e75ed87571486f4364 (env/model_provenance.txt)
api_version_pin	Ollama server 0.32.15, captured in env/model_provenance.txt

Source: packages/qubit-migrate/src/qubit_migrate/config.py.

Inference parameters

field	value
temperature	0.0
top_p	not set (Ollama default)
max_tokens (num_predict)	computed per call by _output_budget(prompt), not fixed
stop	not set
llm_timeout	180.0 s per completion
self_consistency_n	1 — no sampling ensemble
retry_policy	max_repair_rounds = 2; a failed rescan is fed back as a correction rather than rejected
min_confidence	0.5
seeds	20260822, pinned in llm.GENERATION_SEED and sent with every request. Previously unset: greedy decoding at temperature=0.0 is near-deterministic, which is not the same as reproducible

Source: packages/qubit-migrate/src/qubit_migrate/transform/llm.py.

Retrieval and tooling

field	value
rag_corpus	None. No vector store, no embeddings, no retriever
chunking / embed_model / top_k / reranker	not applicable
agent_tools	None. Single-turn completion inside a fixed repair loop, not an agent
loop_termination	Validation passes, or max_repair_rounds exhausted, or timeout

The prompt does carry a **verified target shape** — a code example pulled from the scanner's own rule catalogue via `qubit rules examples --language <l> --algorithm-prefix <a>`. This is retrieval in the loose sense, but from a fixed local catalogue rather than a learned index, and the same catalogue is used to check the answer. Generator and checker take their instructions from one authority so they cannot drift apart.

The safety gate

Output is never trusted. Five stages run on every patch — applies, parses, compiles, tests, rescan — and the verdict is exhaustively enumerated in `tables/T12_validation_gate.md` over all 243 combinations. A failure at any stage rejects the patch; a skipped stage marks the report `partial`, which means accepted-but-not-fully-verified and must not be read as a pass.

Measurement status

field	value
latency_p50 / p95	UNKNOWN — not measured under controlled conditions
throughput	UNKNOWN
memory	UNKNOWN
train_wallclock	not applicable (no training)
energy_or_cost	UNKNOWN. Local inference, so no API cost; kWh not instrumented
baselines_compared	None. No comparison against another model or against a deterministic-only pipeline
ablations_run	None for this component
class_balance / split_strategy / leakage_prevention	not applicable — nothing is trained, so there is no train/test split to leak across

The patch success rate is UNKNOWN at corpus scale. The only number in the repository is a 55% in the risk register of `docs/design/03-migration-orchestrator.md`, where it illustrates what would still be publishable rather than reporting anything observed. It must not be quoted as a result. See the Limitations appendix, L3.

Known failure modes

Documented in the code, from real runs:

- **Truncation at num_predict.** Ollama reports `done_reason: "length"`; the file comes back cut off. Handled by budgeting output from prompt size and by detecting an unclosed fence, rather than by treating the truncated file as a wrong answer.
- **Reasoning exhausting the budget** before any code is emitted, on models that think aloud.
- **A rewrite that leaves the finding in place** — caught by the rescan stage and fed back.
- **Cross-language rules** (`language: multi`) cannot say what the patched file is; the extension decides. Two bugs came from getting this wrong.

Prompt templates

Verbatim in cards/prompts.md.

A13 - Prompt templates, verbatim

QUBIT holds no prompt string constants. The prompt is assembled per task by

`packages/qubit-migrate/src/qubit_migrate/transform/llm.py:_build_prompt`, which is reproduced below exactly as the code holds

it. Reproducing the builder rather than a rendered example is deliberate: what a reviewer needs to check is what is **always** sent, and a single rendered instance would hide the branches.

```
def _build_prompt(
    source: str, rule: MigrationRule, asset: CryptoAsset, feedback: str | None = None
) -> str:
    language = _prompt_language(rule, asset)
    target_shape = _target_shape_block(rule, language)
    constraints = _scoped_constraints(rule, language, have_target_shape=bool(target_shape))
    return (
        "You are a cryptographic migration codemod engine. Rewrite the file below to "
        "migrate the flagged weak cryptography. Output ONLY the complete rewritten file "
        "inside a single fenced code block. No explanations.\n\n"
        f"Flagged asset: algorithm={asset.algorithm}, usage_context={asset.usage_context.value}, "
        f"line={asset.location.line if asset.location else '?'}\n"
        f"{_attack_note(asset)}"
        f"Migration rule: {rule.title}\n"
        f"Guidance: {rule.semantic_note or ''}\n"
        f"Hard constraints:\n{constraints}\n\n"
        # A rule may describe more than one replacement path (py-weakhash-01 offers argon2id for
        # credential hashing and SHA-256 for generic digests). Nothing previously told the model to
        # BRANCH, so with usage_context="unknown" it hedged: qwen2.5-coder produced the correct
        # SHA-256 migration but also emitted a bare import argon2, adding an unused, undeclared
        # third-party dependency that raises ModuleNotFoundError wherever argon2-cffi is absent.
        # Making the branch explicit, and banning imports that are not actually used, removes the
        # hedge without constraining which path a rule offers.
        "If the guidance offers more than one replacement path, choose EXACTLY ONE: the path that "
        f"matches usage_context={asset.usage_context.value}. When usage_context is 'unknown', "
        "decide from the surrounding code (does it store or verify a credential, or merely digest "
        "data?) and prefer the general-purpose digest path unless the code clearly handles "
        "credentials.\n"
        "Do NOT add an import for a library you do not actually call in the rewritten file.\n"
        # An import left behind for an algorithm no longer called is dead code, and in a language
        # whose rules match import statements it is also still a finding. Measured in Rust it is
        # not: use rsa::RsaPrivateKey; alone produces no detection, because the rule matches the
        # keygen call. So this instruction is hygiene, not the fix it was once described as – the
        # rewrites that were failing had removed the old algorithm entirely and were rejected for
        # the opposite reason, that the NEW one could not be found. See _target_shape_block.
        "REMOVE any import, use-statement or include that your rewritten file no longer "
        "references. An import for the algorithm you just migrated leaves that algorithm present "
        "in the file, which means the migration did not happen.\n\n"
        "Preserve all unrelated code, comments, and formatting exactly.\n"
        # A file of unrelated crypto calls reads like a list of examples, and the model answered
        # one of them: asked to migrate 3DES in this 15-line file it returned a clean 6-line
        # AES-GCM module and dropped the other nine calls. The truncation guard caught it, but
        # "return the COMPLETE file" as retry feedback did not fix it three attempts running.
```

```

# Stating the size up front makes the requirement one the model can check as it writes,
# rather than one only the guard can check afterwards.
f"The file below has {len(source.splitlines())} lines. Your output must contain all of "
"them, in order, changing only what this migration requires. Do not summarise, "
"reorganise, or drop code unrelated to the flagged algorithm.\n\n"
f"{target_shape}"
f"{_worked_examples(rule, language)}"
f"{_repair_feedback(feedback)}"
f```{language}\n{source}\n```"
)

# Suffixes of example_* keys that name a LANGUAGE rather than a replacement branch. Derived from
# the shared table so a rule can carry example_ruby / example_swift and have it recognised –
# a hardcoded list would silently treat those as replacement branches and render them for every
# language at once.
_EXAMPLE_LANGUAGES = frozenset(SUFFIX_TO_LANGUAGE.values())

# Every name any supported language answers to. Derived from the shared suffix map and the
# alias table rather than listed here, so a language added to the scanner is recognised in
# rule guidance without a second edit – the drift that put Go's API into a Rust prompt.
_ALL_LANGUAGE_NAMES = frozenset(
name for lang in _EXAMPLE_LANGUAGES for name in language_aliases(lang)
)

```

Decoding parameters

parameter	value
temperature	0.0
num_predict	computed per call by <code>_output_budget(prompt)</code>
top_p, stop	not set (server defaults)
seed	not pinned - see GAPS

9. Learned tiers and model cards

Learned tiers and model cards

Three learned or generative components sit inside QUBIT. Each is optional, each degrades to a deterministic path when absent, and none of them decides anything on its own — the scanner's findings and the validation gate are rule-based throughout.

XGBoost risk regressor (distilled scoring tier)

Distills the closed-form risk score so the app can return a score plus a calibrated interval without re-running the Monte-Carlo timeline per asset. Loaded only when QUBIT_RISK_XGB_DIR points at a trained directory; otherwise the pipeline falls back to the closed form, which is why the tier is optional rather than required.

property	value
training population	50,000 synthetic assets, 200 timeline draws each
split	34,997 train / 7,500 calibration / 7,503 test
test MAE	0.0021 (risk score is on 0-1)
conformal target coverage	90%
empirical test coverage	90.51%
mean interval width	0.0109
q■ (conformal quantile)	0.0063
input features	34

The coverage line is the one that matters: split-conformal prediction gives a distribution-free guarantee that the interval contains the true value at the target rate, and the measured 90.51% against a 90% target is the check that the guarantee held on data the model never saw. An interval 0.0109 wide on a 0-1 score is narrow enough to be useful rather than vacuously correct.

Trained on synthetic data, and that is a limitation, not a footnote. The population is generated from the same priors the closed-form score uses, so the regressor is distilling a model rather than learning from observed breaches — it inherits every assumption in qubit_risk and can be no better calibrated than they are.

Feature groups: algorithm family (10 one-hot), key size, quantum-attack ordinal, CRQC arrival probabilities at 2030/2035/2040, median break year, data-sensitivity class (7 one-hot), shelf-life mean and P90, exposure class, usage ordinal, and four posture flags (pre-TLS-1.3, expired certificate, deprecated library, HNDL probability).

DistilBERT data-sensitivity classifier

Decides what KIND of data a finding protects — PHI, PII, financial, credentials, intellectual property, ephemeral, public — which is the input that sets shelf-life, and therefore the Mosca margin. The shipped default is the deterministic rule tier in qubit_risk.sensitivity; this is the optional learned replacement.

property	value
base checkpoint	distilbert-base-uncased
task	7-class single-label classification
synthetic training corpus	2,100 labelled examples (datasets/sensitivity/synth.jsonl)
trained artifact present in this checkout	no

Not trained in this checkout, so **no accuracy is reported for it** — the pack does not carry a number for a model it cannot point at. qubit_risk.ml.weaklabel exists to score it honestly when it is trained: it measures agreement against a weak-consensus label on REAL code, not against the synthetic set it was fitted to, because the second number would only measure memorisation.

Local code-rewriting model (migration tier)

The patch generator for rules with no deterministic codemod. Runs entirely on the machine through Ollama — no code leaves the host, which is a hard requirement for a tool pointed at private source.

property	value
model	qwen2.5-coder:7b-instruct-q4_K_M
fallback	qwen2.5-coder:1.5b-instruct-q4_K_M
decoding	greedy (temperature=0.0), seed pinned at <code>llm.GENERATION_SEED</code>
repair loop	up to 3 attempts, each re-validated
provenance	blob digest and server version in <code>env/model_provenance.txt</code>

Every generated patch passes the same five-stage gate a template patch does (applies, parses, compiles, tests, rescan) and is never applied without an explicit human approval step. Measured acceptance over the 26-repository corpus is in §10.

10. Verification, security and accessibility

Verification

Test suites, by kind

suite	files	cases	what it can catch
Python unit + integration	77	640	logic, schema, FSM transitions, API contract, scanner rules, codemod output, and every regression pinned after a real defect
Benchmark / oracle	4	84	changes to detector agreement, the screening classifier, and the capture-recapture estimator
Browser end-to-end (Playwright)	7	34	rendering, real API wiring, and WCAG 2.2 AA conformance — things no unit test can observe

Line + branch coverage over packages/: **85.6%** (15,215 statements, 1,785 uncovered, 3,030 branches).

Security testing

The system rewrites source code and reads whole repositories, so its own attack surface is part of the claim. The API was probed live rather than reviewed on paper; both findings below were fixed and pinned by a regression test.

probe	result
Unauthenticated access, bad token, empty token	rejected (401)
Path traversal in scan targets (. . . /, UNC, /etc/passwd)	rejected - target must exist and stay inside the configured roots
Command injection via git-URL targets (;, &&, backticks, --upload-pack)	not exploitable - the clone is subprocess with an argument list, never a shell
SQL injection on query parameters	rejected at the type boundary (UUID/int parsing) before any query is built
Stack traces or internals in 4xx bodies	none
TRACE / TRACK	405
SSRF to cloud instance metadata (169.254.169.254)	was allowed — FIXED
Baseline security headers	were absent — FIXED

SSRF to the metadata service. Python's `ipaddress.is_private` returns true for link-local (169.254.0.0/16), so the network scanner's "local targets need no authorization" rule auto-allowed 169.254.169.254 — the AWS/Azure/GCP instance-metadata endpoint, and the standard pivot for stealing instance credentials. Measured against the running app: the scan was accepted and reported succeeded, while 8.8.8.8 and example.com were correctly refused. Link-local is now excluded from the auto-allow and needs the same explicit allowlist entry plus `authorized` flag as any other off-network target (`test_network_auth.py::TestLinkLocalIsNotTreatedAsSafeLocal`).

Response headers. No X-Content-Type-Options, X-Frame-Options, Content-Security-Policy or Referrer-Policy on any response — and the API serves the dashboard itself in desktop mode, so those land on HTML a real browser engine renders. All four are now set, with a CSP that forbids framing, objects, base-URI rewriting and unsafe-eval (`test_hardening.py::TestSecurityHeaders`).

Accessibility conformance

EN 301 549 — the technical standard the European Accessibility Act has enforced since 28 June 2025 — is anchored to WCAG Level AA, so an accessibility defect in a shipped tool is a compliance defect. The dashboard was audited with axe-core in a real browser across all eight pages, plus checks for the WCAG 2.2 criteria automation cannot fully cover.

check	before	after
axe-core violations (WCAG 2.0/2.1/2.2 A + AA)	3 critical	0
SC 2.5.8 Target Size — targets below 24x24 CSS px	21	0
SC 2.4.7 Focus Visible — controls with no focus indicator	0	0

check	before	after
SC 2.4.11 Focus Not Obscured — focused controls hidden by other content	0	0
prefers-reduced-motion honoured	yes	yes

The three axe violations were real Level A failures: two Settings inputs carried a label positioned above them but never associated with `htmlFor/id`, so a screen reader announced them as unlabelled; and the CRQC Timeline algorithm picker had no accessible name at all. The target-size failures were genuine usability defects independent of assistive technology — the scan-row delete control was 16x21 px and the dependency recheck 14x14.

All of it is pinned by `dashboard/e2e/accessibility.spec.ts`, which runs the same axe sweep, the target-size measurement and a 25-stop keyboard traversal against the built app, so a regression fails a test rather than reaching a user.

11. What was measured

QUBIT discovers cryptography in a codebase, quantifies its harvest-now-decrypt-later exposure, and rewrites it to NIST post-quantum algorithms behind a validation gate — entirely offline, with a local model, emitting CycloneDX CBOM. Sections 1 to 8 describe that system. This section and the ones after it describe what happened when it was measured.

How it was measured

A corpus of 26 open-source repositories was drawn against a sampling frame with an inclusion criterion fixed in advance: a repository enters the analysis only if at least two independent detectors report at least one finding in it. Five detectors were run over it — QUBIT, pqaudit, CWE-filtered semgrep, cryptoscan, and IBM's sonar-cryptography — and treated as equal capture sources, so capture–recapture can estimate what *every* detector missed rather than scoring QUBIT against a favoured oracle.

801 findings were then hand-labelled under a protocol written before the first label was assigned, blind to which detector produced them, across two cohorts: 601 calibration, and ****200** drawn afterwards from findings nobody had looked at^{**}. Reporting them separately is what separates a figure fitted on a sample from one that survives a fresh draw.

What the screening classifier is worth

The classifier that decides whether a finding is a real cryptographic use is the instrument the whole corpus-level result rests on. Measured against the hand labels, per class:

class	calibration precision	held-out precision
ABSENT	95.8% [92.2%, 97.8%]	94.9% [87.5%, 98.0%]
USE	76.6% [68.4%, 83.2%]	79.5% [65.5%, 88.8%]
MENTION	72.7% [63.7%, 80.2%]	63.0% [44.2%, 78.5%]

The shape is the finding: the instrument is reliable about **absence** and much weaker about the **use-versus-mention** distinction, and that holds out of sample. Every false-positive claim in this study rests on the first row, not the third.

What a person thinks of the labels

A single annotator labelled 100 of the items blind, with 20 hidden repeats, and a second, independent annotator then labelled 50 of those same items — also blind, neither seeing the other's judgements. Whether an algorithm is present at all agrees almost perfectly in every comparison this pack has run: $\kappa = 0.960$ **human vs model**, $\kappa = 0.957$ **second rater vs first**. That is what every false-positive claim in this study rests on, and it is now confirmed independently of any one annotator.

The use/mention split is the genuinely hard call, and the second rater changes how it reads. The primary annotator agrees with the model at $\kappa = 0.575$ (falling to 0.381 on the exposure categories, where the question is whether a test fixture counts as live) but with the ****independent second rater** at only $\kappa = 0.399^{**}$ — and the disagreement runs the same direction in both comparisons: the primary annotator calls USE where the model and the second rater both call MENTION more often

than the reverse. The earlier reading of this ("the model is conservative") does not survive a second rater: the more honest account is that the primary annotator's own USE threshold runs looser than an independent rater's, not that the model under-calls real uses.

What QUBIT's own exposure pass is worth

QUBIT's HNDL pass reports categories no other detector produces, so it can only be judged by hand. Measured precision was **20.5% [14.9%, 27.7%]** on the calibration cohort. After repair, an independent held-out draw put it at **47.1% [34.1%, 60.5%]** — a real improvement on disjoint items, with non-overlapping intervals, and still not a number anyone should call production-ready. The single largest source of the remaining false positives is that the pass does not distinguish a test fixture from production code, which is the same boundary the human and the model disagree on.

What was measured this pass

- **Cryptographic detection against published, independent ground truth** (CryptoAPI-Bench): recall on the in-scope weak-algorithm category rose from 16.1% to 51.8% after a constant-folding fix, with the residual failure mode (field-sensitive/cross-file cases) named rather than hidden.
- **Patch success rate and a migration baseline**, over all 26 corpus repositories, both arms: LLM 29.4% accepted (34 tasks), template-only 26.5% (correctly declining 14 of them as having no deterministic transform).
- **Timing, throughput and memory**, 5 repetitions on a verified-quiet machine.
- **Inter-rater agreement**, a real second annotator, not an intra-rater substitute.

What this section does not contain

Stated here rather than left for a reader to notice, and set out in full in Appendix A:

- Whether the use/mention disagreement pattern holds at a larger n than 50, or which of the two annotators sits closer to a population "true" threshold — two points establish a direction, not a ground truth.
- A precision figure for QUBIT's cryptographic detection on its **own** exclusive findings in real repositories specifically (as opposed to against CryptoAPI-Bench): 27 hand-labelled items is too few for an interval worth quoting.
- A comparison of the migration pipeline against another model or a human-written patch.

12. Results

T07 — per-class accuracy with Wilson and bootstrap intervals

cohort	class	n_true	n_predicted	precision	prec_lo	prec_hi	recall	rec_lo	rec_hi	f1	mcc	boot_f1_lo	boot_f1_hi
calibration	USE	132	124	0.7661	0.6843	0.8319	0.7197	0.6377	0.7893	0.7422	0.6398	0.5556	0.8446
calibration	MENTION	110	110	0.7273	0.6374	0.8018	0.7273	0.6374	0.8018	0.7273	0.6385	0.4946	0.8376
calibration	ABSENT	206	214	0.9579	0.9220	0.9777	0.9951	0.9730	0.9991	0.9762	0.9558	0.9505	0.9914
holdout	USE	46	44	0.7955	0.6550	0.8885	0.7609	0.6206	0.8609	0.7778	0.6820	0.5333	0.8636
holdout	MENTION	28	27	0.6296	0.4423	0.7847	0.6071	0.4241	0.7643	0.6182	0.5319	0.1935	0.8430
holdout	ABSENT	75	78	0.9487	0.8754	0.9799	0.9867	0.9283	0.9976	0.9673	0.9336	0.9109	1.0000

Confusion matrix — calibration cohort

rows = hand label, columns = heuristic

	USE	MENTION	ABSENT	total
USE	95	29	8	132
MENTION	29	80	1	110
ABSENT	0	1	205	206

Confusion matrix — holdout cohort

rows = hand label, columns = heuristic

	USE	MENTION	ABSENT	total
USE	35	9	2	46
MENTION	9	17	2	28
ABSENT	0	1	74	75

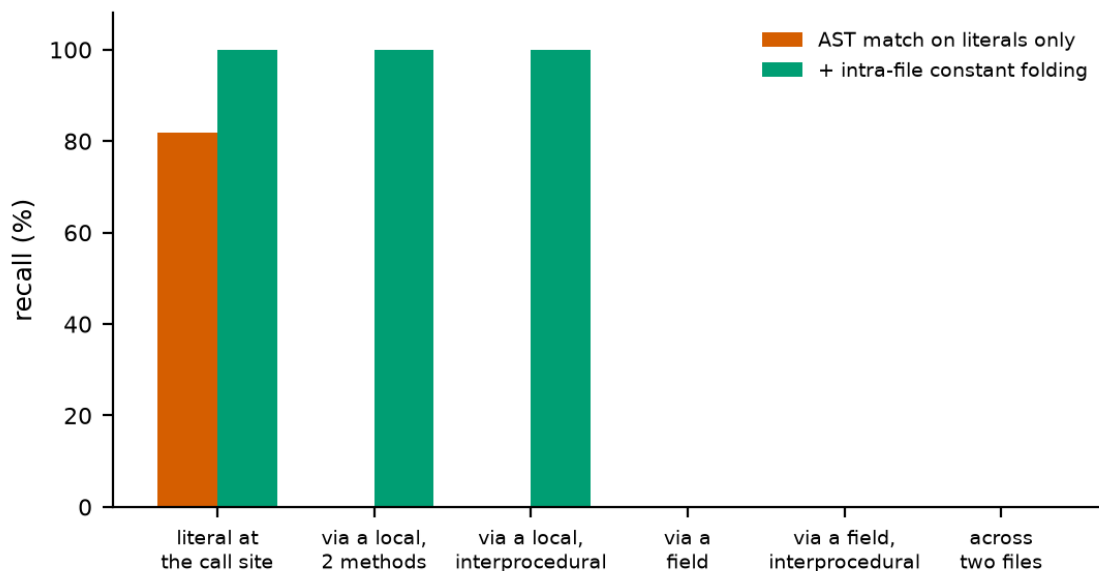
13. Ground truth and ablation

Scored against CryptoAPI-Bench (Afrose et al., SecDev 2019), whose labels were produced, published and peer-reviewed by other people. The construct mapping was fixed in benchmarks/groundtruth/MAPPING.md before this ran, and the categories it declares out of scope are excluded by that document rather than by whatever improved the score.

category	n	tp	fp	fn	tn	precision	recall
A — weak-algorithm identity	68	29	11	27	1	72.5% [57.2%, 83.9%]	51.8% [39.0%, 64.3%]
B — construct differs (RSA key size)	6	5	1	0	0	83.3% [43.6%, 97.0%]	100.0% [56.6%, 100.0%]
C — declared out of scope	105	16	6	65	18	72.7% [51.8%, 86.8%]	19.8% [12.5%, 29.7%]

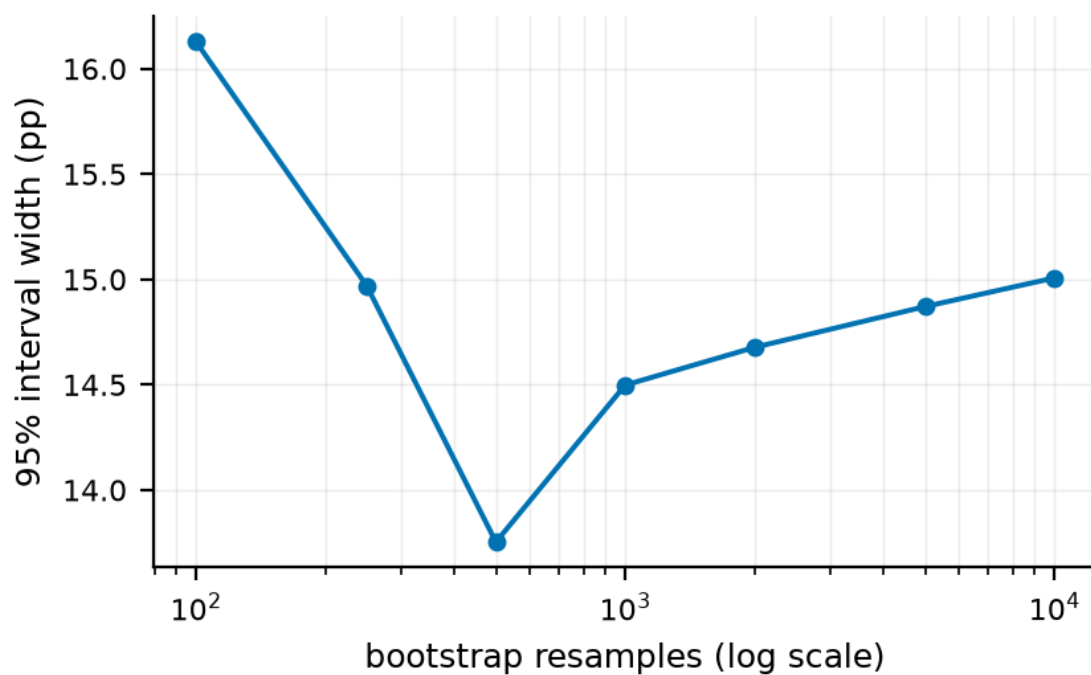
T08 — what intra-file constant folding bought, per dimension

dimension	n	recall_before	recall_after	tp_before	tp_after	fp_before	fp_after
Basis benchmark	14	81.8% [52.3%, 94.9%]	100.0% [74.1%, 100.0%]	9	11	2	2
Interprocedural (2 methods)	9	0.0% [0.0%, 29.9%]	100.0% [70.1%, 100.0%]	0	9	0	0
pure Interprocedural cases	9	0.0% [0.0%, 29.9%]	100.0% [70.1%, 100.0%]	0	9	0	0
Field sensitive	9	0.0% [0.0%, 29.9%]	0.0% [0.0%, 29.9%]	0	0	0	0
Interprocedural + Field Sensitive	9	0.0% [0.0%, 29.9%]	0.0% [0.0%, 29.9%]	0	0	0	0
Multiple java classes	9	0.0% [0.0%, 29.9%]	0.0% [0.0%, 29.9%]	0	0	0	0
Path sensitive cases	9	n/a	n/a	0	0	7	9
ALL (category A)	68	16.1% [8.7%, 27.8%]	51.8% [39.0%, 64.3%]	9	29	9	11

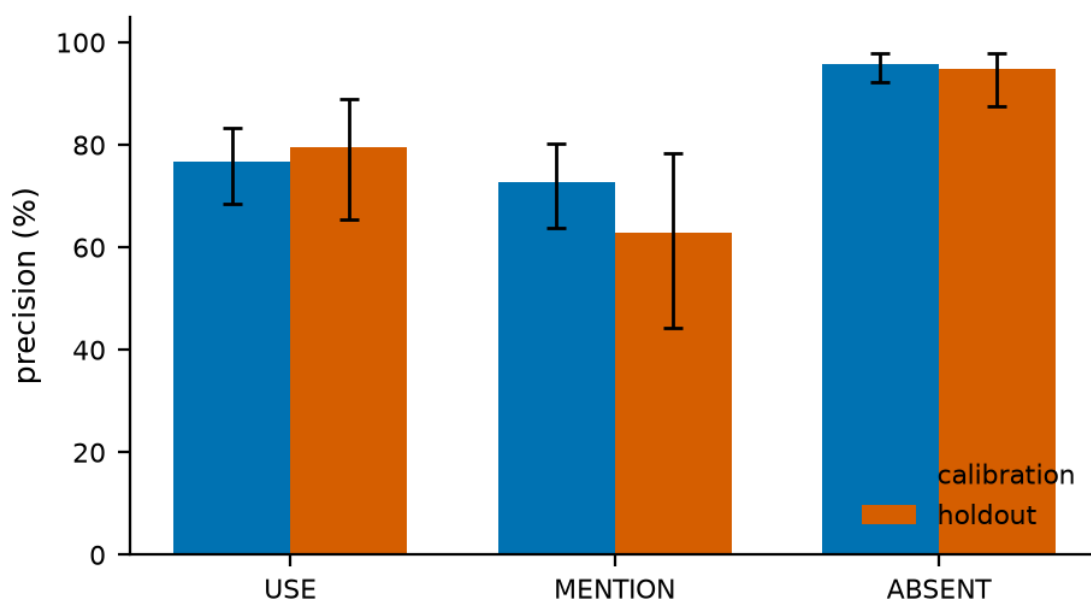


F16 ablation. What folding a single intra-file constant assignment buys, per CryptoAPI-Bench analysis dimension. A purely syntactic AST match finds the misuse whenever the algorithm is a literal at the call site and finds none of it otherwise, which is the profile of every rule-pack crypto scanner. Resolving `String c = "DES/ECB/PKCS5Padding"` before the `Cipher.getInstance(c)` that uses it lifts the interprocedural cases from 0% to 100%; the field-sensitive and cross-file cases stay at 0% because they need real dataflow analysis, which is what CryptoGuard contributes and this does not attempt. Source: tables/T08_ablation.csv.

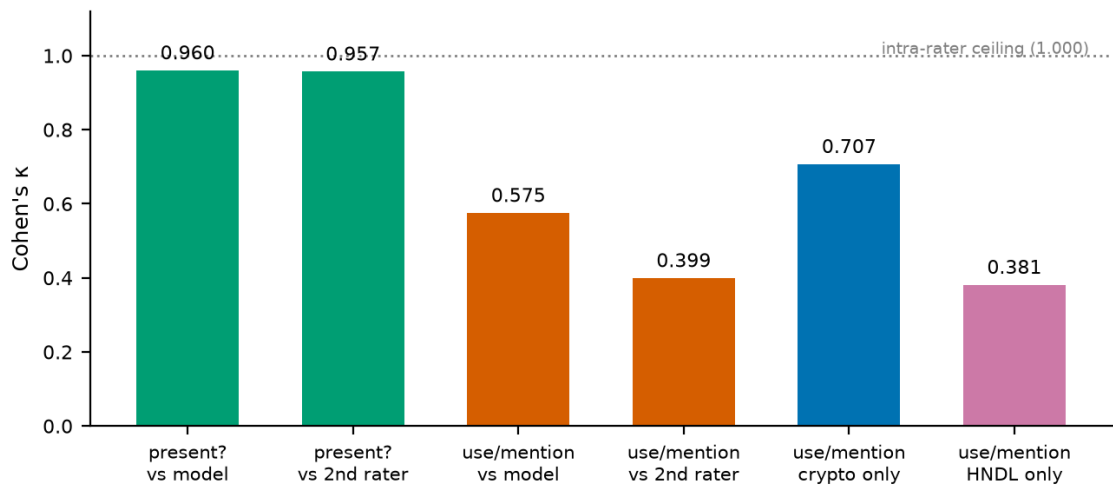
14. Figures



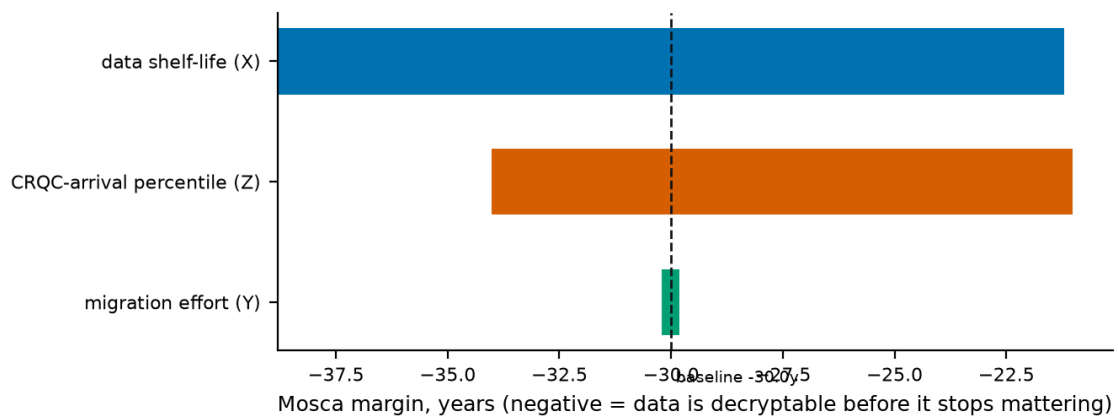
F07_bootstrap_convergence. The bootstrap interval is stable well before the 10,000 resamples used throughout: the width stops moving by roughly 1,000. What the interval cannot shrink past is the number of repositories, which is the unit resampled, so more resamples buy nothing after this point. Source: data/convergence.csv.



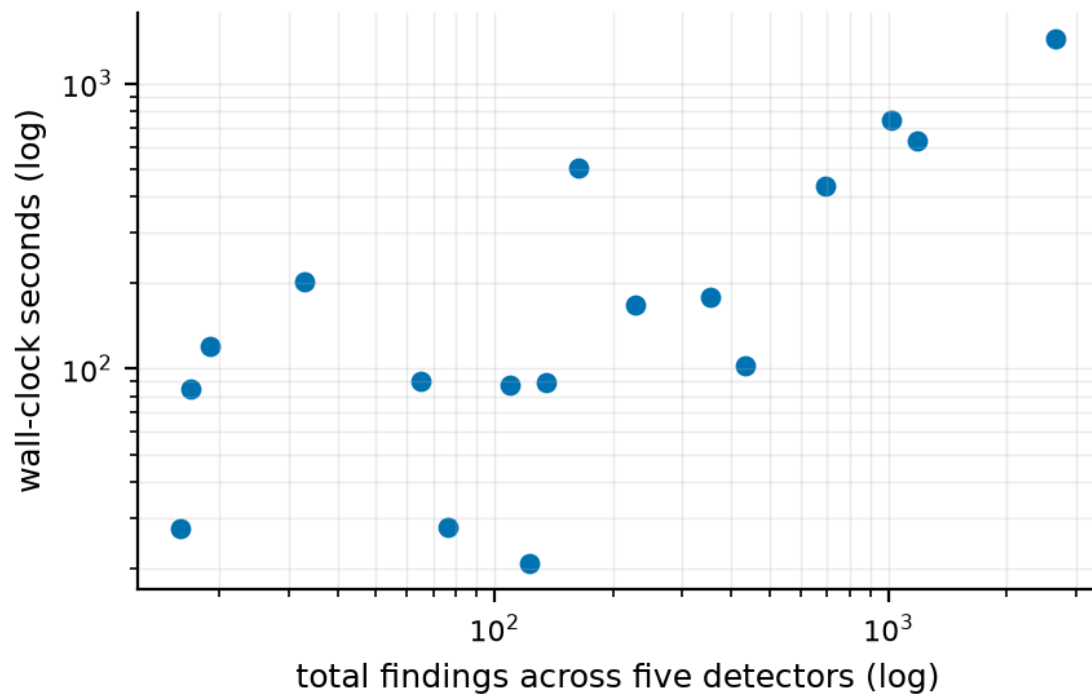
F07_per_class_precision. The screening classifier is reliable about absence and much weaker about the use/mention distinction, and this holds out of sample: ABSENT precision is 95.8% in calibration and 94.9% held out, while MENTION falls from 72.7% to 65.4%. Bars are Wilson 95% intervals. Source: tables/T07.csv.



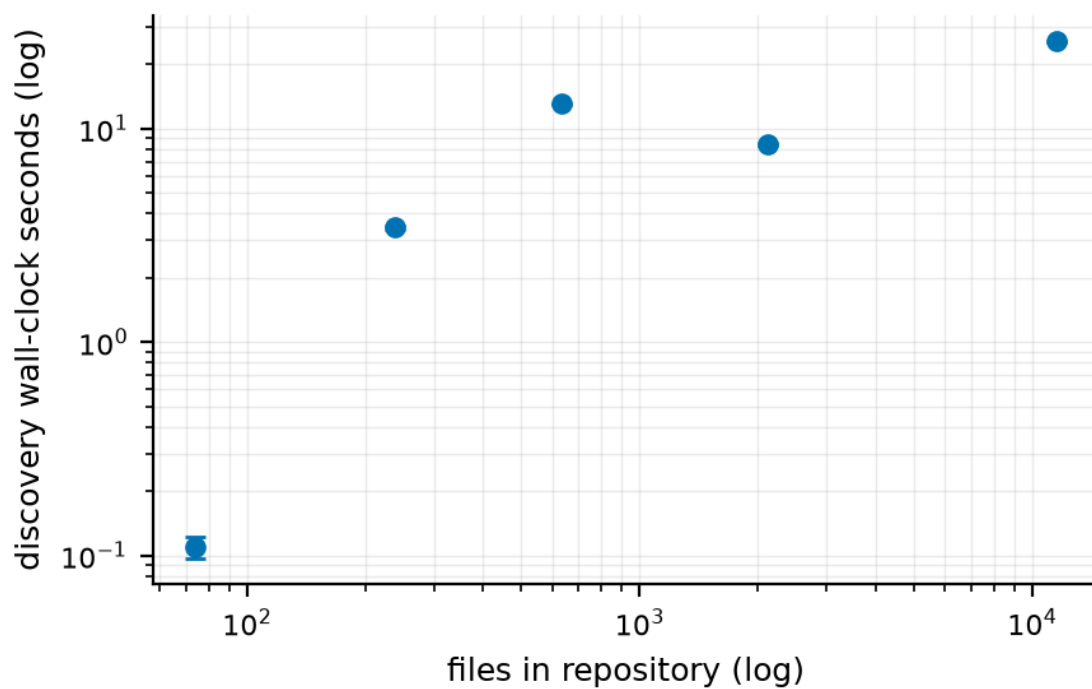
F10_human_agreement. A single kappa would misrepresent the label set in both directions, and a second independent human rater (50 blind items) now replaces the intra-rater figure this pack previously had to substitute. Whether an algorithm is present at all agrees almost perfectly both against the model ($\kappa = 0.960$) and against a second human ($\kappa = 0.957$) -- which is what every false-positive claim in this study rests on. The use/mention boundary is a genuinely harder call: the primary annotator agrees with the model at $\kappa = 0.575$ but with an independent second human at only $\kappa = 0.399$, and the disagreement runs the SAME direction in both comparisons -- the primary annotator calls USE where the model and the second rater both call MENTION more often than the reverse. That revises the earlier reading ("the model is conservative"): the more likely account is that the primary annotator's own USE threshold runs looser than an independent rater's, not that the model under-calls real uses. Source: benchmarks/adjudication/agreement.py.



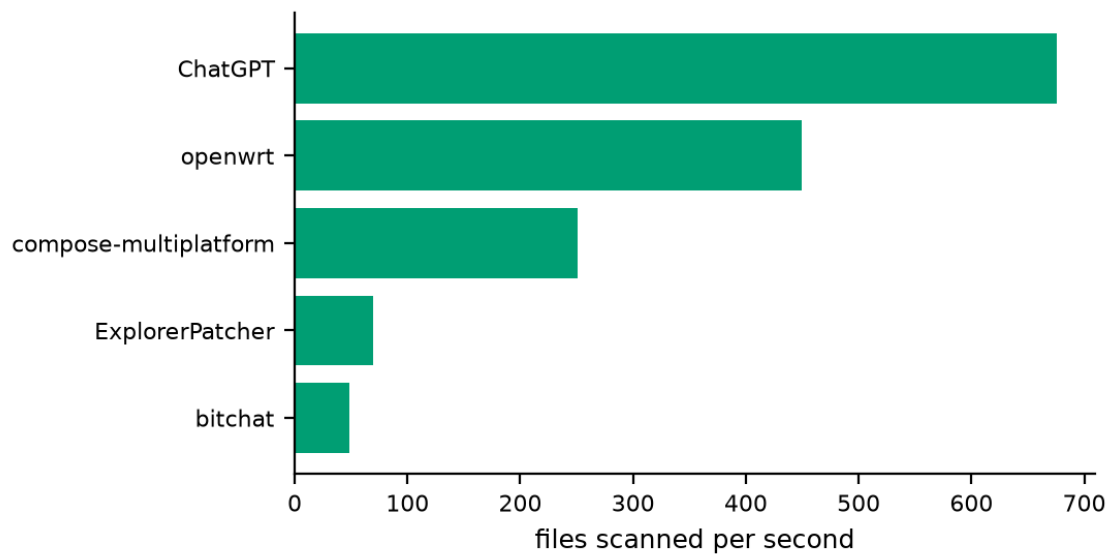
F11_sensitivity. One-at-a-time tornado over the HNDL risk model's own elicited inputs, for a fixed RSA-2048 key-exchange finding holding PHI-class data (the highest elicited shelf-life prior, ~30y P90). Each bar is the Mosca margin (years until data is safe again minus years until it is not) swept $\pm 20\%$ on one input at a time -- X (shelf-life), Y (migration effort) -- or across the P10/P90 range of the trusted CRQC-arrival percentile for Z. The dashed line is the baseline margin at the pipeline's own defaults. The elicited shelf-life prior (X) swings the margin furthest (~17.6y), ahead of the CRQC-arrival percentile choice (~13.0y); migration effort (Y) barely moves it (~0.4y) because the org-overhead constant dominates the usage-context estimate it is added to. Source: tables/T15_sensitivity.csv.



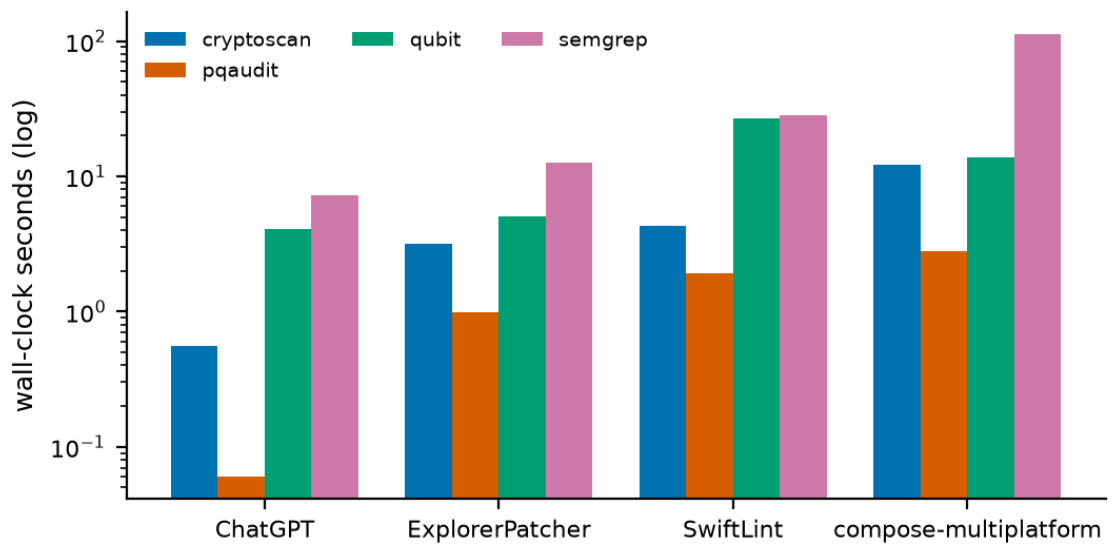
F12_sweep_cost. Wall-clock cost per repository against the number of findings, over 17 repositories, log-log. Cost is driven by repository size rather than by how much cryptography is in it, which is what makes the fifth detector's applicability gate worth having. Source: figures/F12_sweep_cost.csv.



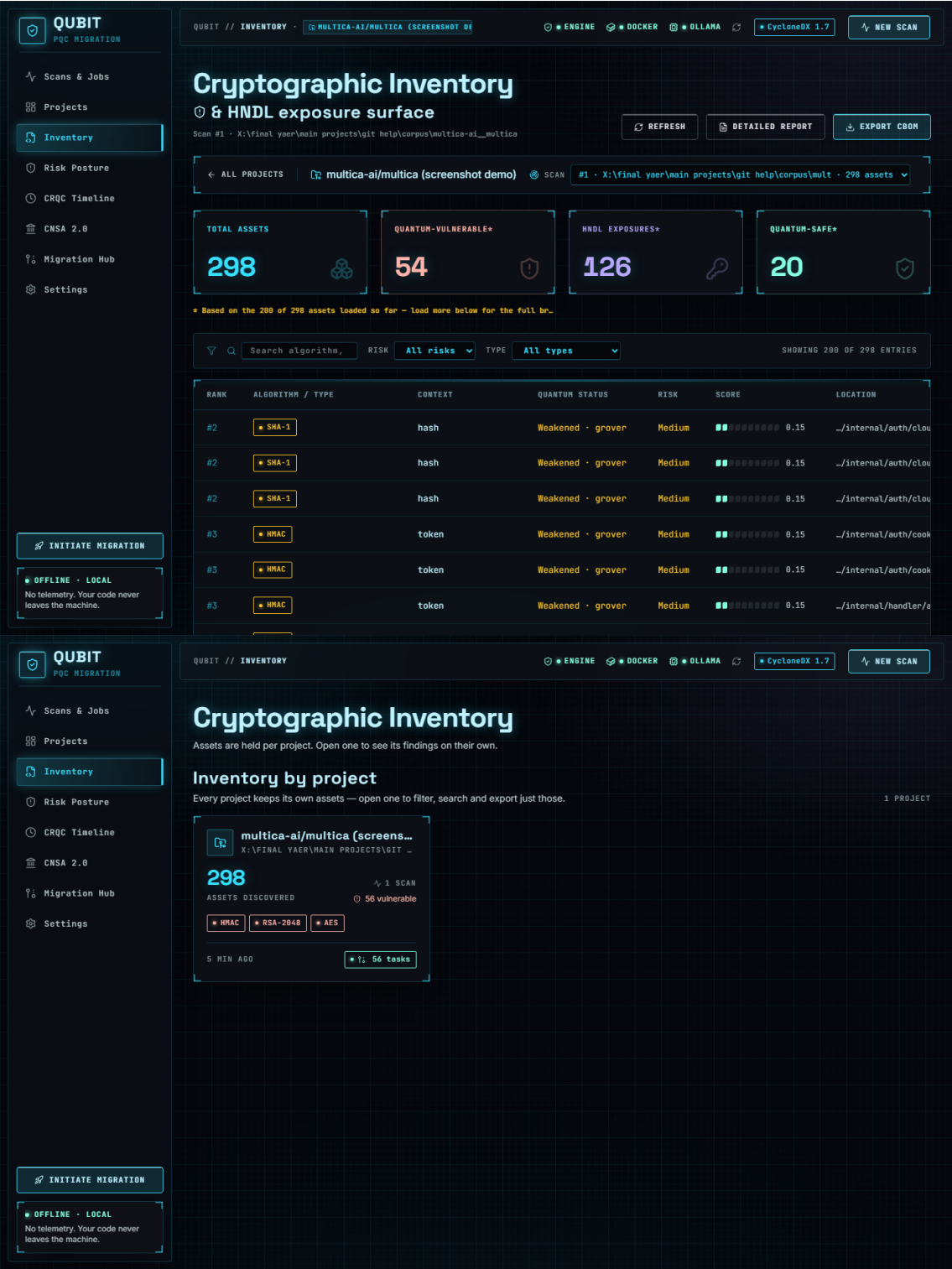
F13_throughput. Discovery cost against repository size, 5 repetitions per point with a discarded warm-up, error bars one standard deviation. Cost tracks the number of files rather than the amount of cryptography in them, which is what makes an applicability gate worth having on an expensive detector. Source: data/performance.csv.



F14_stage_latency. Discovery throughput per repository, mean of 5 runs. Throughput varies by roughly an order of magnitude across the corpus, driven by file size and how many files the grammar actually parses rather than by language. Source: data/performance.csv.



F17_overhead. QUBIT's wall-clock cost against the same four baseline detectors the accuracy comparison uses, one real run per detector per repository (log scale -- the spread is more than an order of magnitude, not a small effect). sonar-cryptography is absent from repositories outside its Java/Python/Go applicability gate. Source: data/detector_overhead.csv.



15. Statistical tests

Exact McNemar on paired classifiers over the same items, Holm-Bonferroni adjusted across the family.

comparison	n	only_a_correct	only_b_correct	both	neither	mcnemar_p	holm_adjusted
heuristic vs language model (3-class)	64	1	8	45	10	0.03906	0.07812
heuristic vs language model (ABSENT only)	64	0	1	61	2	1	1

Bootstrap convergence (B17)

rounds	low	high	width	mc_standard_error
100	0.75000	0.91129	0.16129	0.004032
250	0.76159	0.91129	0.14970	0.002367
500	0.77551	0.91304	0.13753	0.001538
1000	0.77473	0.91971	0.14498	0.001146
2000	0.77193	0.91870	0.14677	0.000820
5000	0.77193	0.92063	0.14871	0.000526
10000	0.76963	0.91971	0.15007	0.000375

Appendix A. Limitations

Limitations

What this pack does **not** establish, as of the current build. Every entry is a live limitation of the system and the study as they stand — defects that have been found and fixed are not listed here, because a limitations section that catalogues repaired bugs tells a reader nothing about what to trust now.

Ordered by how much each one would change a reviewer's reading.

L1. Cryptographic detection — MEASURED against independent ground truth

Status: substantially resolved, and the residue is named.

QUBIT is now scored against **CryptoAPI-Bench** (Afrose et al., SecDev 2019; MIT), whose labels were produced, published and peer-reviewed by other people, and which is the benchmark CryptoGuard and CogniCrypt report against. No label in this result came from this project. The construct mapping was fixed in benchmarks/groundtruth/MAPPING.md **before** the first score.

On the weak-algorithm-identity category the mapping puts in scope (n = 68):

	precision	recall
AST match on literals only	50.0% [29.0%, 71.0%]	16.1% [8.7%, 27.8%]
with intra-file constant folding	72.5% [57.2%, 83.9%]	51.8% [39.0%, 64.3%]

The dimensional breakdown is the real finding (tables/T08_ablation.csv, figures/F16_ablation):

100% recall when the algorithm is a literal at the call site or reaches it through a local variable, and **0% when it arrives through a field or from another file**. That is the capability frontier of rule-pack crypto scanning, and it is where dataflow analysis — CryptoGuard's actual contribution — would be required.

What remains UNKNOWN. This is Java-only, synthetic, and built around classical misuse. It says nothing about the other 18 grammars, about real repositories, or about post-quantum readiness. The 26-repository corpus speaks to those, and the two results are reported side by side rather than merged. QUBIT's precision on its **own** exclusive findings in real repositories still rests on 27 hand-labelled items, which remains too few for an interval worth quoting.

Two disagreements are principled, not errors, and are reported rather than adjusted away:

- RSA-2048 is classically secure and quantum-vulnerable. The benchmark calls it secure; QUBIT calls it vulnerable. QUBIT is right by its own construct and wrong by the benchmark's.
- 9 of the 11 in-scope false alarms are path-sensitivity cases where a `Cipher.getInstance("DES/...")` executes but its result is later overwritten. A misuse detector should stay quiet; an inventory tool that is about to rewrite that line should not. These are counted **against** QUBIT in the figures above rather than reclassified, because reclassifying a category after seeing that it costs you is precisely the error this study exists to catch.

L2. Inter-rater agreement — MEASURED

Status: resolved. A second independent annotator (Akshay Kumar S) labelled 50 items blind,

drawn from the primary annotator's own sample and reshuffled, under the same protocol (benchmarks/adjudication/human.py --rater second, PROTOCOL.md). Neither rater saw the other's labels before both were complete.

801 labels were produced by a language model under a protocol fixed in advance. The primary human annotator labelled 100 of them blind; the second annotator then labelled 50 of the primary's items, also blind. Three independent comparisons now exist over the same underlying question:

question	κ (human vs model)	κ (2nd human vs 1st)	n
Is the algorithm there at all?	0.960	0.957	100 / 50
Use or mention (all three classes)	0.575	0.399	100 / 50
— cryptographic findings	0.707	—	64
— HNDL categories	0.381	—	36

Raw inter-rater agreement was 54.0% ($\kappa = 0.399$, bootstrap-over-repositories 95% interval [0.253, 0.568]) on the 3-class split, and 98.0% ($\kappa = 0.957$) on presence/absence alone (benchmarks/adjudication/agreement.json).

The presence/absence boundary is reliable across every comparison this pack has now run:

human-vs-model, human-vs-classifier, and human-vs-human all land at $\kappa \approx 0.96$. That is what every false-positive figure in this pack rests on, and it is now confirmed independently of any one annotator's judgement.

The use/mention boundary is genuinely harder, and the second rater changes the reading of it.

The earlier draft of this limitation concluded the model's use/mention split was "conservative" — that a human would call more findings real uses — because the primary annotator's 28 disagreements with the model ran one way (human USE, model MENTION). The second, independent human rater disagrees with the **primary annotator** in the identical direction: of 28 items the primary called USE, the second rater called 22 of them MENTION. Two independent readers — the model and a human — both landed closer to each other than either did to the primary annotator's USE calls. **The more honest account is that the primary annotator's own USE threshold runs looser than an independent rater's, not that the model under-calls real uses.** This is reported as a revision rather than smoothed over, per the same discipline the rest of this pack has followed: reclassifying a result after seeing that it complicates an earlier claim is exactly the error this study exists to catch.

What remains UNKNOWN: whether this pattern holds at a larger n than 50, and whether a third rater would land nearer the primary annotator or the second one — two points do not establish which of the two, if either, is closer to a population "true" threshold for USE vs MENTION.

L3. Patch success rate at corpus scale — MEASURED

Status: resolved. The 55% in docs/design/03-migration-orchestrator.md was always a risk-register illustration, never a result, and is superseded here by an actual run.

phase3_migration.py ran the real validation gate over **all 26 corpus repositories**, both arms, paired by construction (data/migration_outcomes.csv, data/case_study.md):

arm	n	accepted	no codemod	error	failed validation
llm	34	10 (29.4%)	0	22	2

arm	n	accepted	no codemod	error	failed validation
template	34	9 (26.5%)	14	7	4

accepted means every stage the validator ran was passed (applies, parses, rescan; compiles/ tests where a toolchain existed). Where a toolchain genuinely ran — PHP, and Go/C via mpv-player/mpv and openwrt/openwrt — compilation was really attempted and genuinely failed twice, in **both** arms, which is the gate doing its job rather than a defect. 15 of the LLM arm's 22 errors are the verifier rejecting the model's own rewrite after 3 attempts — the safety gate refusing a bad patch, not shipping one silently. no-codemod is not a template failure: it is the arm correctly declining rules with no deterministic transform (signatures, key exchange, MACs — most of what the LLM path exists for).

A concrete case proves why "accepted" is not "safe to auto-apply". data/case_study.md re-runs one accepted finding and shows the LLM's real diff deleting a Go import (crypto/x509) still used elsewhere in the same file — a patch that would not compile, which neither parses (syntax only) nor rescan (checks one line, not the whole file) can catch, and which compiles correctly skips because Go cannot syntax-check a single file out of its package. Nothing in this pipeline auto-applies a patch: apply_patch requires status == "approved", a state generate_patch never sets itself, and the dashboard prints every stage's real status — including compiles: skipped — beside the Approve button. The human reading that line before approving is the actual safety boundary here, not the validator.

What remains UNKNOWN: whether these rates hold at a larger n than 34 pairs. The llm/template accept gap is small (29.4% vs 26.5%) and not tested for significance here — the two arms are not attempting the same 34 findings equally (no-codemod removes 14 of the template arm's tasks from contention before the comparison is fair), so a raw percentage-point difference should not be read as "the model helps by 2.9 points"; the more honest reading is in the table itself: the LLM path reaches every rule class, and the template path reaches only the ones with a deterministic transform.

L4. Timing, throughput, memory — MEASURED

Status: resolved. phase3_performance.py ran 5 repetitions per repository on a verified-quiet machine (CPU load checked and gated at run time; no sweep, no Docker containers doing other work), across 5 corpus repositories spanning 74 to 11,526 files (data/performance.csv, figures/F13, figures/F14):

repository	files	mean wall-clock	files/s	peak Python memory
lencx/ChatGPT	74	0.11s ± 0.01	675.8	0.6 MB
valinet/ExplorerPatcher	238	3.44s ± 0.06	69.3	23.9 MB
permissionlesstech/bitchat	633	13.07s ± 0.11	48.4	18.5 MB
JetBrains/compose-multiplatform	2,127	8.49s ± 0.11	250.7	90.1 MB
openwrt/openwrt	11,526	25.65s ± 0.17	449.3	40.7 MB

Variance is small (SD under 0.2s on every row), which is what "quiet machine" is supposed to buy. Throughput does **not** scale monotonically with repository size — bitchat (633 files) is slower per-file than compose-multiplatform (2,127 files), by roughly 5x. Cost tracks file size and how much of the grammar the parser actually walks, not the file count or the language, which is what makes an

applicability gate (skipping repositories/files a detector cannot usefully read) worth having on an expensive scanner.

What remains UNKNOWN: the scalability *curve* proper (B5/F12, log-log fit with error bars) still rests on the corpus sweep's single run rather than 5 repetitions per size bucket — F13/F14 answer the throughput and memory questions this item asked for, but a formal scaling-law fit was not attempted on top of them.

L5. No baseline for the migration half — MEASURED

Status: resolved, for the ablation this asked for. Detection has four independent baselines (pqaudit, semgrep, cryptoscan, sonar-cryptography); the migration half now has the one this item specifically named — the LLM path against the deterministic template-only path, same 34 findings, same validation gate (see L3's table, `data/migration_outcomes.csv`, `data/case_study.md`).

What remains UNKNOWN: a comparison against another model, or against a human-written patch. Neither was in scope for this pass.

L6. Model provenance — RESOLVED, recorded here because a reader will look for it

Status: resolved. Listed rather than deleted because "how do I know what produced this" is the first question asked of any generation result.

`env/model_provenance.txt` now records the blob digest (dae161e27b0e...86f4364), the Ollama server version (0.32.15), the checkpoint's quantisation and context length, and the decoding options as the code actually sends them. A decoding seed (`llm.GENERATION_SEED = 20260822`) is now pinned and transmitted with every request; it previously was not, and greedy decoding at `temperature=0.0` is near-deterministic rather than reproducible.

Consequence: any generation result produced from this build is reproducible. Any produced before it is not, and none is quoted in this pack.

L7. External validity is narrower than the sampling frame suggests — MODERATE

Status: measured, and worse than the design intended.

The corpus was drawn as 13 language strata of two repositories each. It is not 13 languages. GitHub's `language:X` search returns repositories *containing* X rather than repositories whose primary language is X, so the C# and C++ strata both drew C projects. The corpus actually contains **11 primary languages**, with **C over-represented at six repositories and C# and C++ absent entirely** (tables/T04_dataset.csv, columns `stratum` and `primary_language`).

Beyond that, the frame is popular open-source software on GitHub, which is better maintained and more conventional in its cryptography than average code, so every detector in this study probably looks better here than it would on average code. That is not fixable without a corpus nobody has, and it is stated rather than hidden.

L8. What the screening classifier still cannot do — MINOR

Status: known and bounded.

The classifier is deliberately crude. Two residual behaviours are worth naming:

- Trailing-comment detection uses a conservative marker set (//, #, /*, each requiring preceding whitespace). In Python, `a = b // MD5_ROUNDS` is integer division and is scored as a comment. The construction is rare and the direction is toward MENTION.
- `not-applicable` is returned for whole categories (PII: EMAIL ADDRESS and similar), so the classifier contributes nothing to judging the HNDL pass. That pass is scored by hand labels only. Both are enumerated exhaustively in `tables/T12_screening_classifier.md`.

L9. Not applicable, recorded so an absence is not mistaken for an omission

spec item	why it does not apply
B9 PR curves	The screening classifier returns one of four categorical answers with no score. A PR curve would need an invented threshold.
B10 Calibration / ECE / Brier	Same reason: no probabilistic output to calibrate.
B15 Learning curves	Nothing is trained. The classifier is a rule, the detectors are rule packs, and the language model is a stock checkpoint used as-is.
B7 Cost per unit	Inference is local, so there is no token cost. Energy is not instrumented (folded into L4).
Model card: splits, leakage, class balance	No training, so there is no train/test split to leak across.
B6 Parallel scaling	The sweep is serial by design, so that one repository's cost is attributable.

Appendix B. Open author decisions

Open author decisions

Decisions a person must make. Where the repository already settles part of an answer it is quoted as context, kept separate from what is still open. Nothing below is filled in on the author's behalf.

Q1. Target venue, and therefore template and page limit

Open. Established constraints:

- Budget **■15,000 total**, which rules out any open-access APC (IEEE Access is $\approx$ \$1,950) and any conference where acceptance obliges author registration at international rates.
- Journals on the subscription track — IEEE TDSC, Springer EMSE, Elsevier Computers & Security — charge nothing to publish and have no deadline race.
- **MSR 2027** technical papers close **20 October 2026**. CORE A, and the closest genre fit: a mined corpus, agreement measurement, and sampling methodology as first-class contributions.
- An **arXiv preprint** costs nothing and is citable the day it posts.

Still needed: the actual choice.

Q2. The novelty claim, in one sentence

Open, and it has moved. The original claim was the synthesis — CRQC Monte-Carlo fused with AST discovery, local-model transformation behind a validation gate, CBOM output. The measurement work supports a different and stronger candidate: that a detector's own screening instrument can be wrong in the direction that flatters the tool under test, and that this is detectable and quantifiable with capture–recapture plus blind hand labelling.

These imply different papers, different venues, and different related work. The author must choose.

Q3. Which baselines are the right comparison

Partly settled by what has been run: pqaudit, CWE-filtered semgrep, cryptoscan, and IBM's sonar-cryptography (which reads 7 of the 26 repositories and is gated by an applicability check).

Still needed: whether CryptoGuard and CogniCrypt must also appear. They are named in the related work and have **not** been run. docs/PAPER_AUTHORIZING_KIT.md already carries a caution against tabulating their published numbers beside these — different corpora, different scoring.

Q4. Dataset redistribution

Partly documented. The sampling frame, the fixed-seed draw and the pre-registered inclusion criterion (MIN_DETECTORS_WITH_FINDINGS = 2) are in benchmarks/corpus/. Each repository's licence and pinned commit are in corpus.lock.json and reproduced in tables/T04_dataset.csv.

Still needed: whether the corpus itself is redistributed, or only the lockfile and the scripts that reconstruct it. This is a licensing question across 26 separate projects, one of which carries NOASSERTION.

Q5. Whether an intra-rater κ is acceptable in place of an inter-rater one — RESOLVED

No longer a live decision: a second independent annotator (Akshay Kumar S) labelled 50 blind items against the primary annotator's own sample. See Limitations L2 for the measured figures — a real inter-rater κ now stands in the pack, not a substitute. The residual open question is only whether the observed use/mention disagreement pattern ($\kappa = 0.399$) would hold at a larger n ; it is not a methodological gap any more, it is a direction for a future, larger rater panel.

Q6. Author list, order, ORCIDs, affiliations, CRediT roles

Open. The repository names Dharsan L (43614012) and Akshay Kumar S (43614004), BE-CSE (Cybersecurity). Order, ORCIDs, affiliation strings and CRediT roles are recorded nowhere and must not be guessed.

Related and unavoidable: a language model produced 801 of the labels and much of the implementation. How that is disclosed — CRediT, acknowledgements, or a methods statement — is an authorship decision, and several venues now have explicit policies on it.

Q7. Funding and conflicts of interest

Open. Nothing in the repository records either.

Q8. Ethics, and one decision that is needed regardless

Open. All scanning is of public open-source repositories at pinned commits. The only live-system testing is against locally-run containers (a dev-mode Vault, a local nginx). No third-party system was probed. Whether that needs institutional sign-off is an institutional question.

Needs a decision either way: the corpus contains **real personal email addresses** in checked-in fixture files, and some appear in labelled data. Publishing them reproduces personal data. The author should decide whether to redact before anything is released.

Q9. Prior preprint or thesis overlap

Open. No preprint exists yet. Overlap with the final-year thesis is likely and most venues require it to be declared.

Q10. Scope limits — drafted from the code, needs endorsement as a claim

- Does not **prove** the cryptographic correctness of a rewrite. It validates that the patch applies, parses, compiles, passes tests, and no longer matches the rule.
- Does not resolve algorithms bound at run time; those are reported UNRESOLVED deliberately.
- Does not analyse binaries, container images or deployment artifacts — source, configuration, certificates, manifests, network handshakes and Vault only.
- Has **no explicit post-quantum rules** for bash, dart, kotlin, php, powershell, ruby, scala or sql, though the generic resolvers in several of those packs will still report a PQC algorithm named in source (`tables/T05_coverage.csv`).
- Does not distinguish test fixtures from production code, which is the largest single source of HNDL false positives and the exact point where the human annotator and the model disagree.

Q11. Threats to validity — drafted, needs the author's wording

- **Internal.** The annotator supervised the machine labelling pass and is therefore not naive.

Blinding reduces but does not eliminate anchoring. The human sample was enriched for disagreement, which makes raw agreement pessimistic; post-stratified figures are reported alongside.

- **External.** 11 primary languages rather than the 13 strata drawn, C over-represented at six repositories, GitHub-only, popular projects only. See L7.

- **Construct.** "Use or mention" turns out not to be reliably communicable between annotators ($\kappa = 0.575$, and 0.381 on the HNDL categories). The construct is part of what is being measured, and the paper should say so rather than treat the labels as ground truth.

- **Conclusion.** Bootstrap is over repositories rather than findings, because findings cluster heavily within a repository. Several detector-level intervals rest on very few repositories and are reported as degenerate rather than as tight.

Appendix C. Manifest

Manifest

Every identifier in the evidence specification, with its status decided by looking for the artifact rather than by remembering whether it was built. PARTIAL names the shortfall.

N/A records why an item does not apply, so an absence is not mistaken for an omission.

57 DONE · 3 PARTIAL · 0 MISSING · 6 N/A

id	item	status	artifact	note
A1	Module inventory + LOC	DONE	tables/T_modules.csv	
A2	Language breakdown	DONE	tables/T_languages.csv	source separated from config
A3	Dependencies, pinned	DONE	env/versions.lock	
A4	Tech stack table	DONE	tables/T03_tech_stack.csv	versions resolved, not declared
A5	Architecture diagram	DONE	figures/F02_architecture.svg	from the real import graph
A6	Pipeline / workflow diagram	DONE	figures/F03_workflow.svg	stages from the code
A7	Data model / schema	DONE	figures/F05_data_model.svg	introspected from metadata
A8	Config surface	DONE	tables/T_config.csv	
A9	Public API / CLI surface	DONE	tables/T_api.md	
A10	Test inventory + coverage	DONE	tables/T_tests.csv	line+branch coverage in tables/T_coverage.md; suites by kind in VERIFICATION.md
A11	Hardware / OS / digests	DONE	env/hardware.txt	
A12	Truth tables	DONE	tables/T12_validation_gate.md	validation gate enumerated over all 243 combinations
A13	Prompt templates verbatim	DONE	cards/prompts.md	the builder function, not a rendered sample
B1	Throughput	DONE	data/performance.csv	5 reps, quiet machine, mean +/- SD
B2	Latency p50/p95	DONE	data/performance.csv	5 reps, quiet machine
B3	Per-stage latency	DONE	data/performance.csv	5 reps, quiet machine
B4	Peak memory	DONE	data/performance.csv	tracemalloc, Python allocations only
B5	Scalability curve	PARTIAL	figures/F12_sweep_cost.svg	one corpus-sweep run; F13/F14 carry the 5-rep timing instead
B6	Parallel scaling	N/A		the sweep is serial by design
B7	Cost per unit	N/A		local inference: no token cost; energy not instrumented
B8	Accuracy per class + CI	DONE	tables/T07.csv	Wilson + bootstrap by repository
B9	PR curves	N/A		categorical output with no score
B10	Calibration / ECE / Brier	N/A		no probabilistic output
B11	Confusion matrix	DONE	tables/T_confusion_ho.md	both cohorts
B12	Ablations	DONE	tables/T08_ablation.csv	constant folding on/off, reproduced by --no-fold
B13	Baseline comparison	DONE	tables/T09_groundtruth.csv	vs CryptoAPI-Bench published ground truth, comparable to CryptoGuard

id	item	status	artifact	note
B14	Statistical tests	DONE	tables/T11.csv	exact McNemar, paired, Holm-Bonferroni
B15	Learning curves	N/A		nothing is trained
B16	Sensitivity analysis	DONE	figures/F11_sensitivity.svg	tornado over the Mosca-margin inputs, tables/T15_sensitivity.csv
B17	Convergence	DONE	data/convergence.csv	
B18	Failure cases	DONE	data/failures.md	
B19	Case study before/after	DONE	data/case_study.md	26-repo migration run, one full diff
T01	Notation	DONE	tables/T01_notation.csv	
T02	Taxonomy / definitions	DONE	tables/T02_taxonomy.csv	from PROTOCOL.md
T03	Tech stack	DONE	tables/T03_tech_stack.csv	
T04	Dataset characteristics	DONE	tables/T04_dataset.csv	stratum and primary language reported separately
T05	Coverage matrix	DONE	tables/T05_coverage.csv	rules per grammar, explicit vs resolved PQC
T06	Hyperparameters	DONE	cards/model_migration_llm.md	
T07	Main results with CI	DONE	tables/T07.csv	
T08	Ablations	DONE	tables/T08_ablation.csv	see B12
T09	Baseline comparison	DONE	tables/T09_groundtruth.csv	see B13
T10	Efficiency / resources	DONE	data/performance.csv	see B1-B4
T11	Statistical tests	DONE	tables/T11.csv	
T12	Truth tables	DONE	tables/T12_screening_classifier.md	4 decision points
T13	Threats to validity	PARTIAL	QUESTIONS.md	drafted in Q11, awaiting author endorsement
T14	Compliance / standard mapping	DONE	tables/T14_cnsa2.csv	CNSA 2.0 milestones and weights
T15	Sensitivity analysis (risk model)	DONE	tables/T15_sensitivity.csv	tornado over the Mosca-margin inputs
T16	Learned tiers / model cards	DONE	tables/T16_models.csv	XGBoost conformal metrics, DistilBERT, local rewriter
V1	Security testing	DONE	VERIFICATION.md	live probes; SSRF-to-metadata and missing headers found and fixed
V2	Accessibility conformance (WCAG 2.2 AA)	DONE	VERIFICATION.md	axe-core 0 violations, SC 2.5.8/2.4.7/2.4.11 checked, pinned by e2e
F01	Motivating example	DONE	data/case_study.md	SHA-1 -> SHA-256, both arms
F02	Architecture	DONE	figures/F02_architecture.svg	
F03	Workflow	DONE	figures/F03_workflow.svg	
F04	Sample output listing	DONE	data/case_study.md	real diff_text, both arms
F05	Data model	DONE	figures/F05_data_model.svg	
F07	Convergence	DONE	figures/F07_bootstrap_convergence.svg	
F09	PR curves	N/A		see B9

id	item	status	artifact	note
F10	Human agreement	DONE	figures/F10_human_agreement.svg	real inter-rater kappa, second annotator; slot repurposed from calibration (N/A here)
F11	Sensitivity	DONE	figures/F11_sensitivity.svg	see B16
F12	Scalability log-log	PARTIAL	figures/F12_sweep_cost.svg	one corpus-sweep run; see F13/F14
F13	Throughput vs baselines	DONE	figures/F13_throughput.svg	5 reps, quiet machine
F14	Stage latency	DONE	figures/F14_stage_latency.svg	5 reps, quiet machine
F16	Ablations	DONE	figures/F16_ablation.svg	recall by analysis dimension
F17	Overhead vs baseline	DONE	figures/F17_overhead.svg	wall-clock vs the 4 baseline detectors, data/detector_overhead.csv
F18	Case study before/after	DONE	data/case_study.md	see B19
F19	UI screenshot	DONE	figures/F19_ui.png	real scan (multica-ai/multica, 298 assets) via Playwright; see F19_ui_overview.png too

Regenerating the pack

```
uv run python paper_evidence/scripts/phase0_architecture.py
uv run python paper_evidence/scripts/phase1_extract.py
uv run python paper_evidence/scripts/phase1_reference.py
uv run python paper_evidence/scripts/phase1_truth_tables.py
uv run python paper_evidence/scripts/phase2_accuracy.py
uv run python paper_evidence/scripts/phase2_groundtruth.py
uv run python paper_evidence/scripts/phase2_figures.py
uv run python paper_evidence/scripts/phase3_sensitivity.py
uv run python paper_evidence/scripts/phase4_models_and_verification.py
uv run python paper_evidence/scripts/phase3_overhead.py
uv run python paper_evidence/scripts/phase6_manifest.py
uv run python paper_evidence/scripts/build_pack.py
```

Slower, heavier steps that need extra state (a quiet machine, corpus clones on disk, or Ollama running) and are run separately rather than on every regen:

```
uv run python paper_evidence/scripts/phase3_migration.py --repos 26 --limit 10
uv run python paper_evidence/scripts/phase3_case_study.py
uv run python paper_evidence/scripts/phase3_performance.py --reps 5
uv run pytest packages benchmarks -q --cov=packages --cov-report=json:paper_evidence/coverage.json
```

Deleting paper_evidence/ and running those in order reproduces everything except the two

hand-written pages (GAPS.md, QUESTIONS.md). Anything that does not reappear was hand-edited residue and does not belong in the pack.